

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Bachelor as Bilingual Informatic Engineering

BACHELOR THESIS

**Learning from synthetic images generated by deep
convolutional generative adversarial networks**

**Author: Carlos Vega de Alba
Advisor: Miguel Ángel García García**

May 2025

Learning from synthetic images generated by deep convolutional generative adversarial networks

Author: Carlos Vega de Alba

Advisor: Miguel Ángel García García



**Video Processing and Understanding Lab
Escuela Politécnica Superior
Universidad Autónoma de Madrid
Mayo 2025**

Trabajo parcialmente financiado por el Ministerio de Ciencia e Innovación del Gobierno de España bajo el proyecto PID2021-125051OB-I00 (HarvestingVisualData) (2022-2025).



Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© 20 de mayo de 2025 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, nº 1
Madrid, 28049
Spain

Carlos Vega de Alba

Learning from synthetic images generated by deep convolutional generative adversarial networks

Carlos Vega de Alba

C\ Francisco Tomás y Valiente Nº11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

Acknowledgements

First, I want to thank my advisor Miguel Ángel García García for proposing this topic, letting me continue developing my interest in such an incredible field as deep learning, and always answering all my questions in record time.

Thanks to my family for giving me the opportunity to study what I wanted and for always guiding me along the way, showing me that the best path to take is the one where you enjoy every step. Finally, thanks to my girlfriend for always walking beside me.

Resumen

En la actualidad, el aprendizaje automático basado en imágenes es una herramienta fundamental en diversas aplicaciones como la visión artificial, el reconocimiento facial y los sistemas médicos de diagnóstico asistido. Sin embargo, una de las mayores limitaciones es la escasez y calidad de los conjuntos de datos etiquetados, así como el costo asociado a la recopilación y anotación manual de imágenes reales. Ante esto, han surgido métodos basados en redes generativas adversarias (GANs) que permiten generar imágenes sintéticas realistas capaces de complementar o reemplazar parcialmente los conjuntos de entrenamiento convencionales.

Este trabajo se estructura en tres etapas principales, aplicadas por separado sobre dos subconjuntos específicos de ImageNet: Imagenette e Imagewoof. En la primera etapa, se realizará una evaluación comparativa del desempeño de diferentes modelos de clasificación como ResNet, VGG, AlexNet, SqueezeNet y MobileNet entrenados exclusivamente con imágenes reales de estos subconjuntos. En la segunda etapa, se estudiará en detalle el proceso de generación de imágenes sintéticas utilizando una arquitectura ligera y sencilla basada en redes GAN convolucionales profundas (DCGAN), con el fin de entender mejor cómo ocurre esta generación a partir de los subconjuntos mencionados. Tras evaluar las imágenes generadas por estas redes DCGAN, los resultados obtenidos no eran de alta calidad por lo que se decidió expandir el proyecto para buscar otras alternativas GAN para lograr mejores resultados, aunque no fuera necesario ya que no estaba en los objetivos del proyecto. Finalmente, en la tercera etapa, se utilizará la red generativa preentrenada BigGAN, entrenada previamente sobre el conjunto completo de ImageNet, para generar imágenes sintéticas de alta calidad visual. Estas imágenes sintéticas generadas por BigGAN serán utilizadas para entrenar desde cero los clasificadores mencionados anteriormente, con el objetivo de evaluar si estos modelos son capaces de generalizar correctamente la clasificación sobre imágenes reales nunca antes vistas.

El objetivo central de este estudio es evaluar si los clasificadores, entrenados únicamente con imágenes generadas sintéticamente mediante GANs, son capaces de reconocer y clasificar correctamente imágenes reales, además de estudiar cómo es el proceso de generación de imágenes sintéticas.

Palabras clave

Clasificación de imágenes a partir de imágenes reales, Clasificación de imágenes a partir de imágenes sintéticas usando BigGan, Imagenet, imagenette, imagewoof, generación de imágenes sintéticas usando DCGAN, ResNet, VGG, AlexNet, SqueezeNet, MobileNet.

Abstract

Currently, image-based machine learning is an essential tool in various applications such as computer vision, facial recognition, and medical diagnostic assistance systems. However, one of its major limitations is the scarcity and quality of labeled datasets, as well as the high costs associated with the manual collection and annotation of real images. To address this issue, methods based on Generative Adversarial Networks (GANs) have emerged, capable of generating realistic synthetic images that can partially complement or replace conventional training datasets.

This study is structured into three main stages, applied separately to two specific subsets of ImageNet: Imagenette and Imagewoof. In the first stage, a comparative evaluation of various classification models such as ResNet, VGG, AlexNet, SqueezeNet, and MobileNet, trained exclusively with real images from these subsets will be tested. In the second stage, the generation of synthetic images will be studied in detail using a lightweight and simple architecture based on Deep Convolutional Generative Adversarial Networks (DCGAN) to better understand the synthetic generation process from these subsets. After evaluating the images generated by these DCGAN networks, it was found that the results were not of high quality, so it was decided to expand the project to explore other GAN alternatives to achieve better results, even though this was beyond the project's original objectives. Finally, in the third stage, the pre-trained generative network BigGAN, previously trained on the full ImageNet dataset, will be employed to generate high-quality synthetic images. These synthetic images generated by BigGAN will be used to train the previously mentioned classifiers from scratch, aiming to evaluate whether these models can correctly generalize and classify real images they have never seen before.

The central objective of this study is to evaluate whether classifiers trained solely with synthetically generated images using GANs can accurately recognize and classify real images and additionally to examine the synthetic image generation process.

Keywords

Image classification using real images, Image classification using synthetic images generated by BigGAN, ImageNet, Imagenette, Imagewoof, synthetic image generation using DCGAN, ResNet, VGG, AlexNet, SqueezeNet, MobileNet.

Index

Acknowledgements	i
Resumen	iii
Abstract	v
Index	vii
List of figures	x
List of equations	xi
List of tables	xi
1 Introduction	1
1.1. Motivation	1
1.2. Objectives	2
2 State of the art	3
2.1. Introduction to Neural Networks	3
2.1.1. What is a neural network	3
2.2. Forward Pass	4
2.3. Backpropagation	5
2.3.1. Loss function	5
2.3.2. Gradient descent	7
2.3.3. Fitting Models	8
2.4. Different types of hidden layers	8
2.5. ImageNet	8
2.5.1. Imagenette	9
2.5.2. Imagewoof	10
3 Design and development	11
3.1. Development environment	11
3.2. Classifiers and hyperparameters	11
3.3. Data Augmentation	12
3.4. Classifiers trained with real images	13
3.5. DCGAN. Training from scratch	14
3.5.1. Models Structure	14
3.5.2. Training Models	15
3.5.3. Upscaling image Dimensions	15
3.5.4. DCGAN images results	16
3.6. GANs. Pretrained Models	16
3.6.1. Synthetic Images Examples	17
3.6.2. Choosing the best GAN	18
3.6.3. Is BigGan the best GAN model?	19
3.7. Classifiers trained with fake Images	19

4	Results	20
4.1.	Classifiers trained with real images	20
4.1.1.	Loss evolution	20
4.1.2.	Accuracy progression	21
4.1.3.	Best results	21
4.1.3.	Training time	23
4.2.	DCGAN. Training from scratch	23
4.2.1.	DCGANs training configurations chosen	23
4.2.2.	Loss evolution	24
4.2.3.	Accuracy progression during training	25
4.2.4.	Time trained	28
4.2.5.	DCGANs best model	28
4.2.6.	Showing best DCGANs image creation process	29
4.3.	Classifiers trained with fake Images.....	32
4.3.1.	Data augmentation in synthetic images.....	32
4.3.2.	Loss evolution	33
4.3.3.	Accuracy evolution.....	34
4.3.4.	Training time	36
4.4.	Comparison	36
4.5.	Cost of training models	37
5	Conclusions	38
5.1.	Future work	39
	Bibliography.....	40
	Appendices	41
1	Code	43
2	DCGAN Results	63

List of figures

Figure 2.1. Simple fully connected neural network.	3
Figure 2.2. Activation Functions.	4
Figure 2.3. Depicting neural networks.....	5
Figure 2.4. Depicting neural networks.....	6
Figure 2.5. Adaptive moment estimation (Adam).	7
Figure 2.6. Imagenette image dimensions.	9
Figure 2.7. Imagenette image examples.	9
Figure 2.8. Imagewoof image dimensions.	10
Figure 2.9. Imagewoof image examples.	10
Figure 3.1. Real images of Imagenette with transformations.	13
Figure 3.2. Real images of Imagewoof with transformations.	13
Figure 3.3. DCGAN Generator 64x64 architecture.	14
Figure 3.4. DCGAN Generator (second configuration) image creation results compared to real images.	16
Figure 3.5. BigGan-deep-256 Image Creation.....	17
Figure 3.6. BigGan-deep-512 Image Creation.....	17
Figure 3.7. SAGAN Image Creation.	17
Figure 3.8. SNGAN Image Creation.	17
Figure 3.9. Biggan-deep-256.	18
Figure 3.10. ImageNet 256x256 images FID models rank.	19
Figure 3.11. StyleGAN-XL image creation for ImageWoof with truncation of 0.4.....	19
Figure 4.1. Classifiers trained with real images loss.....	20
Figure 4.2. Classifiers trained with real images accuracy progression.....	21
Figure 4.3. Accuracy per class of classifiers trained with synthetic images Imagenette.	22
Figure 4.4. Accuracy per class of classifiers trained with synthetic images Imagewoof.....	22
Figure 4.5. Training time for Imagewoof and Imagenette, classifiers trained with synthetic images.	23
Figure 4.6. DCGAN loss evolution during training Imagenette.....	24
Figure 4.7. DCGAN loss evolution during training Imagewoof.	25
Figure 4.8. DCGAN accuracy evolution during training Imagenette.....	26
Figure 4.9. DCGAN accuracy evolution during training Imagewoof.	27
Figure 4.10. DCGANs time trained.	28
Figure 4.11. DCGAN synthetic image creation evolution for Imagenette.	30
Figure 4.12. DCGAN synthetic image creation evolution for Imagewoof.....	32
Figure 4.13. GAN synthetic images of Imagenette with transformations.	32
Figure 4.14. GAN synthetic images of Imagewoof with transformations.	33
Figure 4.15. Classifiers trained with synthetic images loss.....	33
Figure 4.16. Classifiers trained with synthetic images accuracy progression.	34
Figure 4.17. Accuracy per class of classifiers trained with synthetic images Imagenette.	35
Figure 4.18. Accuracy per class of classifiers trained with synthetic images Imagewoof.....	35
Figure 4.19. Training time for Imagewoof and Imagenette, classifiers trained with real values.	36
Figure 4.20. Iberdrola energy consumption.	37

List of equations

Eq. 2.1.....	5
Eq. 2.2.....	6
Eq. 2.3.....	6
Eq. 2.4.....	6
Eq. 2.5.....	6
Eq. 2.6.....	6
Eq. 2.7.....	7
Eq. 2.8.....	7

List of tables

Table 2.1. Imagenette classes frequency.	9
Table 2.2. Imagenette image dimensions.	9
Table 2.3. Imagewoof classes frequency.....	10
Table 2.4. Imagewoof image dimensions.	10
Table 3.1. GAN synthetic images FID and IS.	18
Table 4.1. Classifiers trained with real images accuracy.	21
Table 4.2. Best Classifier accuracy with DCGAN images Imagenette.	28
Table 4.3. Best Classifier accuracy with DCGAN images Imagewoof.....	29
Table 4.4. Classifiers trained with synthetic images accuracy.....	34
Table 4.5. Real and Synthetic classifiers comparison.....	36

Introduction

In this Bachelor's Thesis, supervised learning and synthetic image generation using deep convolutional generative adversarial networks (GANs) are studied for image classification tasks using convolutional neural networks (CNNs). Specifically, the GAN architectures DCGAN and BigGAN are used and evaluated with different CNNs such as ResNet34, VGG13, MobileNet_v2, SqueezeNet1_1, and AlexNet. Additionally, a comparative analysis is conducted between metrics obtained by training these CNNs with real images versus synthetic images generated by GANs.

1.1. Motivation

Deep networks have improved exponentially during the last years, making them able to produce better results each time and become the most powerful tool in many different fields such as computer vision, natural language processing, speech and audio processing.

Deep Learning demands a large amount of data in order to achieve exceptional performance. Unfortunately, many fields do not have big amounts of data, and it is normally manually labeled by humans, which is a costly process as it is time-consuming and error-prone. Having sufficient data is the first step toward any successful and trustworthy DL application.

As a response to these data dependencies, there is an increasing trend of creating models that can produce synthetic data to be able to train other models that could solve the problems of data scarcity [7]. There have been multiple architectures discovered for these generative models and one of the most popular ones is the GAN method where two models are trained at the same time, one being a generator that produces synthetic data and the other one a discriminator receives both real and synthetic images and judges if they are either real or not, making the models train each other as the generator would generate progressively better samples and the discriminator would become progressively better judging.

Using synthetic data to train models in order to reduce costs or solve the problem of data scarcity may seem like a solution but substituting the real data with synthetic does not guarantee that our models become able to achieve good results when they are evaluated with real data.

This project will study how good 5 different classifiers models can become in classifying real image data between 10 classes. Firstly, the ten classes would be very different from each other and then the difficulty will increase as the classes will become similar to each other. Then it will be explored how synthetic data is generated from scratch and lastly the possibility of training the same 5 classifiers with this synthetic data and judging the results by comparing them to the classifiers trained with real images.

1.2. Objectives

The main objective of this work is to evaluate the effectiveness of training convolutional neural networks using synthetic images generated by GANs compared to real images. The specific objectives to be achieved are detailed below.

O-1.– Study and select the most relevant CNN architectures for image classification tasks: ResNet34, VGG13, MobileNet_v2, SqueezeNet1_1, and AlexNet.

O-2.– Study synthetic image generation methods using deep convolutional generative adversarial networks, specifically DCGAN.

O-3.– Implement the selected convolutional networks in PyTorch and train them with real images from the Imagenette and Imagewoof datasets.

O-4.– Generate synthetic images using pre-trained models such as BigGAN and evaluate their visual quality and diversity.

O-5.– Train the selected CNNs exclusively using synthetic images generated by GANs.

O-6.– Compare and analyze the performance of models trained with synthetic images versus those trained with real images.

State of the art

In the *State of the Art* section, we will explore the foundations of neural networks, their structure and training methods, review different model types, and examine benchmark datasets like ImageNet and its subsets, Imagenette and Imagewoof.

2.1. Introduction to Neural Networks

Artificial intelligence, or **AI**, refers to build systems that simulate intelligent behavior. It has multiple approaches, some based on logic, search or probabilistic reasoning. Then we have **Machine Learning** that is a subset of AI and learns to make decisions by fitting mathematical models to observed data. Due to the popularity it has achieved thanks to all the new programs that use Machine Learning, it has become almost synonymous the term AI. **Deep neural networks** (or **deep networks** for short) are a type of machine learning, and they are the most powerful tool in many fields such as writing, image recognition/creation, speech recognition...

The enterprises investment over the past 15 years to now has been exponentially growing and we now can encounter them in our day-to-day life in many different fields, for example in tools such as ChatGPT, Gemini, Chatbots, Google Translate, Google Lens, that are very useful for many tasks.

2.1.1. What is a neural network

Neural Networks are inspired by how human brains work [2] and, like our brains, they learn by practice. If we take a look at a more detailed view of the structure, we can distinguish three main parts.

- The input, that would be what we provide to the neural network.
- Hidden layers, that would perform some mathematical calculations to the given input.
- The output that would be the results that we obtain after the transformations are applied.

For a better understanding let's look at a simple visual example of a neural network.

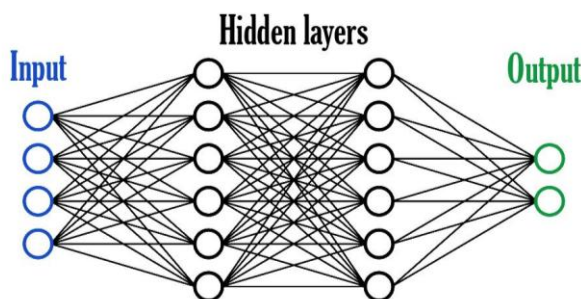


Figure 2.1. Simple fully connected neural network.

To be able to understand how they work we will break them down. First, we have the simplest unit, the **neuron** (the circles of the image). We can think of the neuron as just a value between 0 and 1. Then we have **layers** that are a group of neurons (the group of vertically aligned neurons). A neural network can have multiple layers, and each layer can have multiple neurons inside. The number of layers and neurons per layer would be a design choice. For example, in Figure 2.1, we can see four neurons in the input layer, six neurons

on each hidden layer and two neurons on the output layer. There are several types of layers we can choose to train our neural network depending on what we want our neural network to do, in fact, obtaining good results would directly depend on this choice. For now, we can simplify them to the one shown in the picture that is a **fully connected layer**, where every neuron inside a layer is connected to all the neurons of the previous layer and all the neurons of the following layer. Each connection between neurons as we can see in Figure 2.1 is made by an arrow, these arrows are called **weights**, and each weight (represented by each arrow) is just a numerical value that will be updated during the training to get the desired results.

2.2. Forward Pass

Now that we know the structure of a neural network it will be described in detail the process of producing an output (result) from an input, a process known as the **forward pass**. To be able to get the value of one neuron we will also have 2 more elements, the **activation function** and the **bias** and the process will be as follows.

Step 1: Weighted Sum Calculation. The neuron takes the inputs coming from neurons in the previous layer and multiplies each input by its respective weight. It then adds all these weighted values together, including an additional term called a bias, which acts similarly to an offset that helps the neuron activate or deactivate more effectively.

Mathematically, this step can be represented as follows: $z = (w_1 \times x_1) + (w_2 \times x_2) + \dots + (w_n \times x_n) + b$

z is the weighted sum (also called the neuron's input). $w_1, w_2, \dots, w_n$ are the weights connecting the previous neurons to this neuron. $x_1, x_2, \dots, x_n$ are the outputs (values) of neurons from the previous layer. b is the bias term.

Step 2: Activation Function. After obtaining the weighted sum (z), it is passed through an activation function. This activation function introduces non-linearity to the network, allowing the neural network to model complex patterns and relationships in the data. Common activation functions include ReLU, Sigmoid, or Tanh.

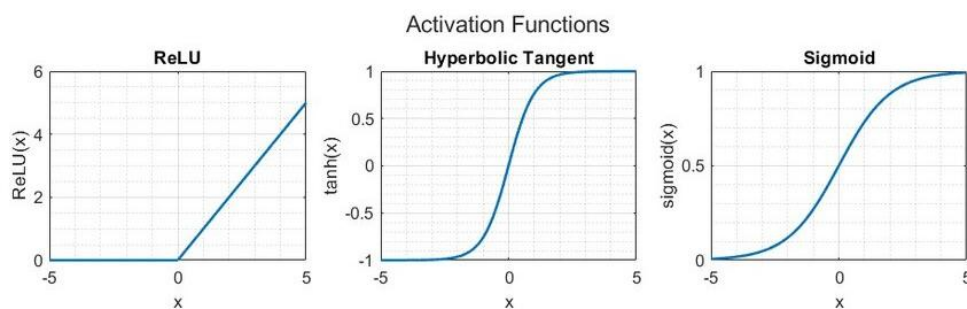


Figure 2.2. Activation Functions.

This step is mathematically represented as: $a = f(z)$. Where a is the neuron's output (activation) and $f(z)$ is the activation function applied to the weighted sum.

What we just did is called **forward pass**, input data moves forward through the network, neuron by neuron, layer by layer, calculating activations until reaching the output layer. A practical example could be providing an image of an animal to our neural network and choosing if it is either a cat or a dog. Let's see a visual example of how everything would take place in Figure 2.3.

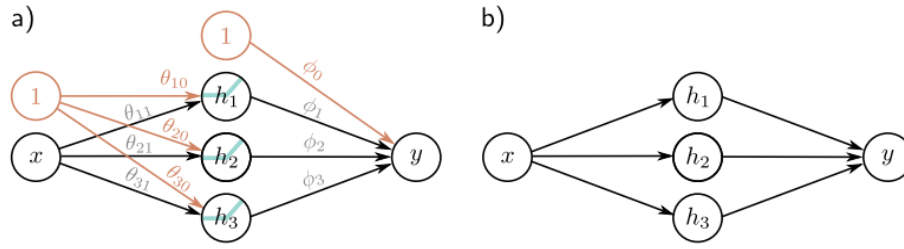


Figure 2.3. Depicting neural networks.

a) The input x is on the left, the hidden units h_1 , h_2 , and h_3 in the center, and the output y on the right. Computation flows from left to right. The input is used to compute the hidden units, which are combined to create the output. Each of the ten arrows represents a parameter (intercepts in orange and slopes in black).

Each parameter multiplies its source and adds the result to its target. We introduce additional nodes containing ones (orange circles) to incorporate the bias (offsets) into this scheme. b) More typically, the intercepts, ReLU functions, and parameter names are omitted; this simpler depiction represents the same network.

2.3. Backpropagation

As we have already seen how the neural networks produce results from input data by the forward pass, now it is time to explore the **backward pass**, that is the algorithm that makes them able to learn from each result produced. As we just saw, the neural networks are just multiple neurons and weights connected to each other following some logical design choice. What distinguishes a model that can produce good results from another that has the same architecture but is not able to do what is meant to, is just the mathematical values of these neurons and weights. So, the task becomes clear, it is necessary to find the best possible values for these neurons and weights. We could define the backpropagation as the algorithm that from a produced result is able to determine how much each neuron and weight contributed and depending on the mismatch of the answer, readjust those values in order to produce better results next time. But how are we able to measure how good is the current mapping of our given input to our generated output. This is done by using a loss function, also known as cost function. This explanation will be based on the Understanding Deep Learning book see [1].

2.3.1. Loss function

The loss function measures how far the model's predictions are from the real values, and the goal of training a model is to try to reduce this number as much as possible. We will first explore how the loss value is calculated before showing how to minimize it. The recipe to construct a loss function is as follows.

First, we have to think of our model as computing the conditional probability $P(y|x)$ over the possible outputs y given the input x . How we compute this conditional probability would directly depend on the given task. For example, for **regression** the model predicts continuous probability distributions over outputs, and for **classification**, the model predicts discrete probabilities for each class. Before we proceed to define the specific forms of possible loss functions used in neural networks, it is essential to understand clearly the rationale behind the Maximum Likelihood criterion, a foundational concept in statistical modeling and deep learning, **Maximum Likelihood Criterion**. The Maximum Likelihood criterion involves choosing model parameters in such a way that the observed training data becomes most probable under the assumed probability distribution of the outputs. Mathematically, this can be expressed as Equation 2.1.

$$\hat{\phi} = \arg \max_{\phi} \prod_{i=1}^I Pr(y_i | f[x_i, \phi]) \quad \text{Eq. 2.1.}$$

Being: ϕ : Model parameters, $\hat{y}$: Predicted output, y : Actual observed output, x : Input data, θ : Distribution parameters predicted by model, $P(y|x)$: Conditional probability of observing output given input, I : Total number of data points, $f[x_i, \phi]$: Neural Network forward pass.

This product form is derived under two key assumptions. **Identically Distributed:** We assume the probability distribution of outputs is the same for each data point. **Independence:** The conditional distributions of the outputs given their inputs are independent, allowing us to multiply their probabilities. However, the direct application of this criterion can lead to numerical instability since multiplying many probabilities (can end in very small results as probability numbers are between 0 and 1) may lead to computational underflow. To overcome this practical limitation, we maximize the logarithm of this likelihood, which transforms the product into a sum, like in Equation 2.2.

$$\hat{\phi} = \arg \max_{\phi} \sum_{i=1}^I \log [Pr(y_i | f[x_i, \phi])] \quad \text{Eq. 2.2.}$$

Maximizing this **log-li** is ically equivalent to maximizing the original likelihood since the logarithm is a strictly monotonically increasing function. Conventionally, optimization problems are framed as minimization problems. Therefore, we typically minimize the negative of this log-likelihood in Equation 2.3.

$$\hat{\phi} = \arg \min_{\phi} \left[- \sum_{i=1}^I \log Pr(y_i | f[x_i, \phi]) \right] \quad \text{Eq. 2.3.}$$

This criterion is precisely what we refer to as the **negative log-likelihood loss**, commonly used as a foundational loss function in deep learning. Finally, once the model parameters have been determined, the model defines a probability distribution over possible outputs rather than directly predicting a single outcome. To obtain a concrete prediction or point estimate during inference, we usually select the output that maximizes this predicted probability distribution as in Equation 2.4.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{Eq. 2.4.}$$

Two widely used loss functions are **Mean Squared Error (MSE)** for regression tasks and **Cross-Entropy (CE)** for classification problems. MSE calculates the average squared differences between predictions and actual values. Cross-Entropy measures the model's accuracy in classification by evaluating predicted probabilities against true class labels. We can visualize both formulas in Equation 2.5 and 2.6.

$$\hat{y} = \arg \max_y Pr(y | f[x, \hat{\phi}]) \quad \text{Eq. 2.5.}$$

$$CE = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad \text{Eq. 2.6.}$$

Choosing the appropriate loss function greatly affects the effectiveness and accuracy of a model's predictions.

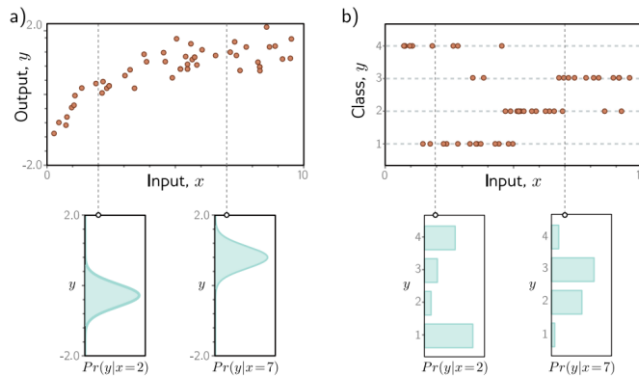


Figure 2.4. Depicting neural networks.

Predicting distributions over outputs. a) Regression task, where the goal is to predict a real-valued output y from the input x based on training data $\{x_i, y_i\}$ (orange points). For each input value x , the machine learning model predicts a distribution $Pr(y|x)$ over the output $y \in R$ (cyan curves show distributions for $x = 2.0$ and $x = 7.0$). The loss function aims to maximize the probability of the observed training outputs y_i under the distribution predicted from the corresponding inputs x_i . b) To predict discrete classes $y \in \{1, 2, 3, 4\}$ in a classification task, we use a discrete probability distribution, so the model predicts a different histogram over the four possible values of y_i for each value of x_i .

2.3.2. Gradient descent

The loss function indicates how far the predictions made are from reality, but how can we make our predictions closer to reality? This is done by **Gradient Descent**. This algorithm updates the model weights to reduce the loss function. This is done in two steps: the first step is to compute the derivatives of the loss with respect to the parameters, which basically determines how much each parameter contribute to the final result.

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}. \quad \text{Eq. 2.7.}$$

And then, update the parameters with a chosen criterion α that is known as the learning rate, and will determine the magnitude of adjustment we apply in every update.

$$\phi \leftarrow \phi - \alpha \cdot \frac{\partial L}{\partial \phi}, \quad \text{Eq. 2.8.}$$

One problem we usually find is that we will often encounter **local minima** where our gradient is zero, and the loss increases if we move in any direction. To address these challenges, methods such as **random weight initialization** and **adaptive optimizers** (Momentum, Adam, RMSprop) are frequently employed.

Random weight initialization ensures that training starts from varied initial parameter values, significantly decreasing the chance of immediately falling into suboptimal regions.

On the other hand, **adaptive optimizers**, such as **Momentum**, help parameter updates by incorporating a fraction of previous updates into the current step. Specifically, Momentum accumulates past gradients to maintain a running average, thereby accelerating updates in consistent directions and reducing oscillations. This technique helps the optimization escape shallow local minima or saddle points by overcoming flat or challenging regions in the loss landscape. An illustrative example comparing the trajectories of standard Gradient Descent and Gradient Descent with Momentum across the loss landscape is provided in Figure 2.5.

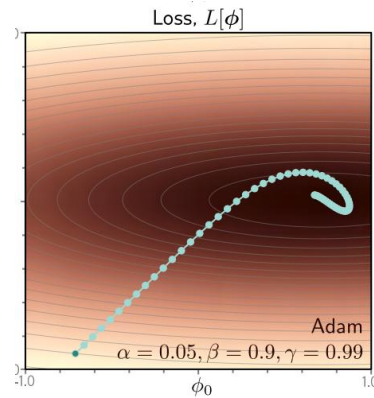


Figure 2.5. Adaptive moment estimation (Adam).

Adam initially moves rapidly toward regions of lower loss, efficiently bypassing oscillations and shallow local minima. As it approaches the minimum, the path becomes smoother, converging stably towards a low-loss area

α represents the **learning rate**, controlling the step size of each update during parameter optimization. β denotes the **momentum parameter**, which controls how much of the previous gradient information influences the current update. γ represents an additional hyperparameter in adaptive optimizers (like Adam or RMSprop) controlling the rate at which past squared gradients decay or diminish. This parameter ensures that recent gradient magnitudes have more influence, allowing the optimizer to adapt quickly to the current landscape.

2.3.3. Fitting Models

Fitting our models is the process of finding the adjustments that produce the best results and therefore there are some common issues we will encounter when we train models in DL

Overfitting and Underfitting: Overfitting occurs when a model performs very well on training data but poorly on unseen data. The model memorizes the training set rather than learning general patterns. Underfitting occurs when a model fails to capture the underlying trend in the data, resulting in poor performance even on training data. Typically caused by models that are too simple or insufficiently trained. **Early Stopping:** A technique used to prevent overfitting, training stops when performance on a validation dataset starts to degrade, even if the training loss continues to decrease. It ensures the model maintains better generalization by not fitting noise in training data. **Learning Rate Scheduler.** Method to dynamically adjust the learning rate during training. Helps improve convergence and model performance. Usually done by reducing the learning rate as we keep training to first learn the basic patterns and then more sophisticated ones. **Dropout:** A regularization method that temporarily removes neurons during training to reduce dependencies between units. It effectively prevents overfitting by forcing the model to learn more robust and generalized patterns. **Batch Size:** Refers to the number of training examples processed before updating the model's parameters. Smaller batch sizes introduce more noise into gradient estimates, helping to escape local minima, while larger batch sizes provide more stable gradient estimates but may require more memory.

2.4. Different types of hidden layers

Fully Connected (Dense) Layers: In these layers, each neuron is connected to every neuron in the preceding and succeeding layers. They are versatile and suitable for various tasks, as they can capture complex relationships in data. However, due to the high number of parameters, they are prone to overfitting and can be computationally intensive. **Convolutional Layers:** These layers apply filters to local regions of the input, effectively capturing spatial hierarchies and patterns such as edges and textures. Convolutional layers are fundamental in Convolutional Neural Networks (CNNs) and are particularly effective for image and spatial data processing. **Pooling Layers:** Often used in conjunction with convolutional layers, pooling layers reduce the spatial dimensions of the data by summarizing features, which helps in reducing computational load and controlling overfitting. Common pooling operations include max pooling and average pooling. **Recurrent Layers:** Designed to handle sequential data, recurrent layers such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRUs) maintain internal states to capture temporal dependencies. They are widely used in tasks involving time series, speech recognition, and natural language processing. **Normalization Layers:** These layers, including batch normalization and layer normalization, standardize the inputs to a layer, which stabilizes and accelerates the training process. They help with mitigating issues like internal covariate shifts and can improve the generalization of the model. **Attention Layers:** Attention mechanisms allow the model to focus on specific parts of the input, assigning different weights to different inputs. This is especially useful in tasks where context is important, such as in natural language processing and machine translation.

Selecting the appropriate combination of these hidden layers is critical for building effective neural network models. The choice should be guided by the nature of the data and the specific requirements of the task at hand.

2.5. ImageNet

ImageNet is a large-scale image database containing over 14 million labeled images spanning thousands of diverse categories, making it a crucial resource for training and evaluating deep learning models in computer vision tasks [5]. However, the extensive size and complexity of ImageNet often impose significant computational demands, prompting the creation of smaller, more manageable subsets designed for efficient experimentation. Two widely adopted subsets, Imagenette and Imagewoof, will be utilized in this project. An important characteristic of these datasets is their inherent **real-world variability**, encompassing images captured from various perspectives, under different lighting conditions, backgrounds, scales, and angles, as well as instances with multiple objects or partial occlusions. This variability closely mirrors real-life visual challenges and significantly increases the complexity involved in accurately classifying images and training image generator models.

2.5.1. Imagenette

Imagenette is a curated subset of 10 classes from the full ImageNet dataset, selected to be visually distinct and easier to classify. It was created to help researchers and developers test and benchmark neural networks quickly, especially when working with limited computational resources.

The classes that are contained are: *tench*, *English springer*, *cassette player*, *chain saw*, *church*, *French horn*, *garbage truck*, *gas pump*, *golf ball*, *parachute*

The frequency and total images of each class are shown in Table 2.1.

Table 2.1. Imagenette classes frequency.

Dataset	tench	English springer	cassette player	chain saw	church	French horn	garbage truck	gas pump	golf ball	parachute	TOTAL
Training	963	955	993	858	941	956	961	931	951	960	9469
Validation	387	395	357	386	409	394	389	419	399	390	3925
											13394

As we can see, the classes are well distributed, having almost the same number of images per class, this will facilitate the training process as the model will not tend to choose a class just based on what is more likely due to having more samples.

In Figure 2.6. and Table 2.2. we can see the distribution of the dimension size.

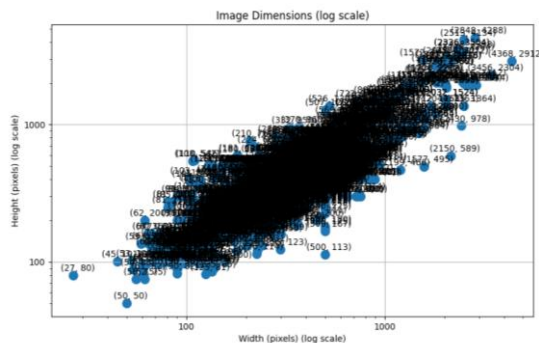


Table 2.2. Imagenette image pixels dimensions.

	Lowest	Highest	Most common	Average
Width	27	4368	375	408
Height	50	4288	500	471

Figure 2.6. Imagenette image dimensions.

From this data we can interpret that the images vary between high and low resolutions, having an average value of around 400x400 pixels which is a perfect value because the classifiers that we will use have 224x224 pixels which is smaller so resizing images will not lose a lot of quality.

Some examples of the dataset images in Figure 2.7.



Figure 2.7. Imagenette image examples.

2.5.2. Imagewoof

Imagewoof is another 10-class subset of ImageNet, but it includes only dog breeds. It is intentionally more challenging than Imagenette due to the subtle visual differences between the breeds, making it useful for testing a model's fine-grained classification performance.

The classes that are contained are: *Australian terrier*, *Border terrier*, *Samoyed*, *Beagle*, *Shih-Tzu*, *English foxhound*, *Rhodesian ridgeback*, *Dingo*, *Golden retriever*, *Old English sheepdog*

The frequency and total images of each class in Table 2.3.

Table 2.3. Imagewoof classes frequency.

Dataset	tench	English springer	cassette player	chain saw	church	French horn	garbage truck	gas pump	golf ball	parachute	Total
Training	941	942	932	580	949	943	949	928	921	940	9025
Validation	409	408	418	224	401	407	401	422	429	410	3929
											12954

And the distribution of the dimension size in Figure 2.8. and Table 2.4.

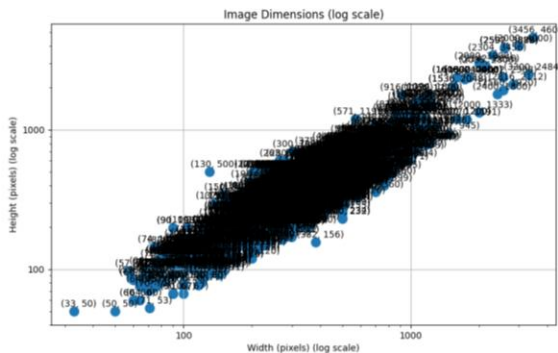


Table 2.4. Imagewoof image pixels dimensions.

	Lowest	Highest	Most common	Average
Width	33	3456	375	397
Height	50	4608	500	470

Figure 2.8. Imagewoof image dimensions.

The classes frequency and images dimension distribution are similar, so the same conclusions we get for both, in Figure 2.9. are shown some examples of the dataset images.



Figure 2.9. Imagewoof image examples.

Design and development

In this section, we describe in detail the design considerations, methodologies, and procedures followed in the development of this project. We begin by outlining the criteria for selecting datasets, specifically Imagenette and Imagewoof, as well as the rationale behind choosing the convolutional neural network architectures: ResNet, VGG, AlexNet, SqueezeNet, and MobileNet for image classification tasks. Additionally, we explain the configuration and training process of our generative models, including the DCGAN and the pretrained BigGAN network, highlighting the motivations for employing separate generators and discriminators per class. Furthermore, we describe the experimental setup, defining essential hyperparameters such as the noise vector dimension, learning rates, image resolution, and the strategies for updating model weights. This comprehensive overview provides clarity regarding each step undertaken, ensuring reproducibility and understanding of our methodological choices.

3.1. Development environment

This work has been carried out using the Python programming language in a Conda environment with Jupyter notebooks. Specifically, the Python framework called **Pytorch**, developed by **Meta AI**, was used to create machine learning projects involving neural networks, and to load the pretrained GANs using the **pytorch_pretrained_biggan** library. This library allows us to create datasets, construct neural networks, optimize their weights through training, and finally save them. This is possible because **Pytorch** utilizes tensors. A tensor is a data structure similar to arrays, which can be executed and stored on the GPU. These tensors represent the inputs, outputs, and network parameters. Furthermore, these tensors are optimized for automatic differentiation, a critical step for gradient descent and backpropagation during network training.

Typically, during network training, we prefer to execute operations in parallel on the GPU rather than on the CPU. This preference arises because GPUs contain many more cores, albeit at lower speeds than CPUs, making them more suitable for parallel task execution and thereby reducing training time. To perform parallel computations on an NVIDIA GPU, **Pytorch** leverages CUDA support. CUDA is a parallel computing platform and programming model developed by NVIDIA.

Most of the functions in the code such as train, evaluation, save or load, used in this project, have been written from scratch in order to have a bigger control of the training flow and for educational purposes.

In our specific case, we used the Windows 11 operating system, an Intel i9 13900KF CPU at 3, 64GB of RAM, and an NVIDIA GeForce RTX 4070 Super GPU with 12GB GDDR6X memory. For the installation of CUDA, version 11.8 was downloaded from NVIDIA's website. Afterwards, the pytorch version used was **Pytorch 2.0.1+cu118**.

3.2. Classifiers and hyperparameters.

For this project, 5 classifiers provided by Pytorch are chosen: **ResNet34**, **VGG13**, **MobileNet_v2**, **SqueezeNet1_1** and **AlexNet**. Then we will need to choose some **hyperparameters**.

- **Batch size** After multiple tries and monitoring of the GPU VRAM by the locally installed program *TechPowerUp GPU-Z*, the maximum number of images we can safely use to be able to load all the image information in the GPU memory and maximize the GPU efficiency is different for the classifiers and the

DCGAN. To set this value carefully is fundamental because we also need to load the model to the GPU to train, apart from the data. If we exceed the VRAM capacity, we will not encounter any issue but it will result in not being able to train the model as fast as it could due to having multiple 'page fault' as all the data used would not be in the VRAM of the GPU and therefore spending most of the training time moving/removing data from/to the VRAM.

For the classifiers the batch size was set to 50 and VGG13 has the most neurons and weights, so it has been the limiting model to increase the batch size. Classifying the real images we will encounter that this limit slightly exceeds the VRAM limit only for VGG13 as it has the biggest architecture and causes a longer training time needed. For the classifiers with synthetic images this would not happen and therefore we will be able to analyze the extra cost in training time if we exceed the VRAM capacity. For the DCGAN it was increased to 256 due to the light architecture it has.

- **Loss functions:** Using Cross-Entropy Loss is ideal for classifiers involving multiple classes (e.g., 10 classes) because it internally applies a softmax function, measuring how well predicted probabilities match true labels, thus ensuring numerical stability and efficiency in multi-class classification tasks. Conversely, Binary Cross-Entropy Loss is well-suited for binary classification tasks, such as distinguishing real from fake images in a DCGAN, since it measures the difference between predicted probabilities (after sigmoid activation) and actual binary labels, clearly indicating model accuracy and providing effective gradients for training.

- **Adam Optimizer:** Adam (Adaptive Moment Estimation) is an excellent choice for optimizing classification models due to its adaptive nature, efficiently managing the learning rates of individual parameters by accounting for both momentum and variance in the gradients. This adaptability allows Adam to effectively handle sparse gradients and noisy data, commonly encountered in real-world datasets. By dynamically adjusting step sizes, Adam reduces the need for extensive hyperparameter tuning, ensuring robust and fast convergence across various neural network architectures, making it an optimal choice for initial training phases and rapid experimentation. It will be used for classifiers and DCGAN models.

- **CosineAnnealingLR Scheduler:** Using the CosineAnnealingLR scheduler complements Adam by systematically adjusting the global learning rate following a smooth cosine decay. This approach helps the model smoothly transition from aggressive learning (larger steps) during initial training phases, toward, more conservative and precise updates near convergence. Consequently, it improves final accuracy, prevents overshooting optimal minima, and encourages the model to settle into stable, well-generalized solutions, particularly beneficial during the late stages of training in complex classification scenarios. The chosen value for the project is set to the learning rate divided by 100.

- **Learning rate:** The initial learning rate was specifically selected for each model due to their different architectures by manual training several times. The values chosen are: ResNet34, VGG13 and Alexnet had a lr of 0.0001, SqueezeNet a lr of 0.0003 and MobileNet a lr of 0.0005.

- **Number of epochs:** The number of epochs has been set to 300, by manually searching for the best value, what seems to be a perfect parameter to make the model learn without overfitting.

3.3. Data Augmentation

As described in the state of the art, data augmentation is a fundamental technique to help model generalization instead of memorizing the training samples and prevent overfitting. In this project we used data augmentation and it improved the results obtained. The data augmentation pipeline used for the training set was as follows:

For the training set, it starts with a **RandomResizedCrop** transform, scaling images randomly from 40% to 100% of the original size while keeping a fixed aspect ratio and resizing them to 224x224 pixels using bicubic interpolation. This helps the model become robust to changes in object size and location. Next, a **RandomHorizontalFlip** with a 50% probability was applied to promote orientation invariance. The **AutoAugment** with ImageNet policy was then used, which intelligently combines multiple transformations to enhance diversity in training samples. To further increase robustness, **ColorJitter** was incorporated to slightly alter brightness, contrast, saturation, and hue, simulating varying lighting conditions. A mild **GaussianBlur** was incorporated to simulate different levels of image clarity and a random sharpening operation (**RandomAdjustSharpness**) was applied with a probability of 50% to enhance edge details occasionally. Finally, the images were converted to tensors and normalized using ImageNet's mean and standard deviation to ensure consistent data distribution for training. These augmentation choices significantly contributed to the improved generalization and robustness of our model.

It is very important that these transformations just mentioned were applied exclusively to the training dataset to preserve the original validation photos untouched. For the validation set, a simple, consistent, and deterministic preprocessing pipeline was applied: images were resized directly to 224 pixels on the shortest side, followed by a center crop to precisely 224x224 pixels. Subsequently, images were transformed into tensors and normalized using ImageNet's predefined mean and standard deviation. This ensured a fair and unbiased evaluation of the model's true performance by testing it solely on original, unaltered images. Figures 3.1. and 3.2. illustrate examples of the transformations in the real images datasets for both databases, showing two samples per class.

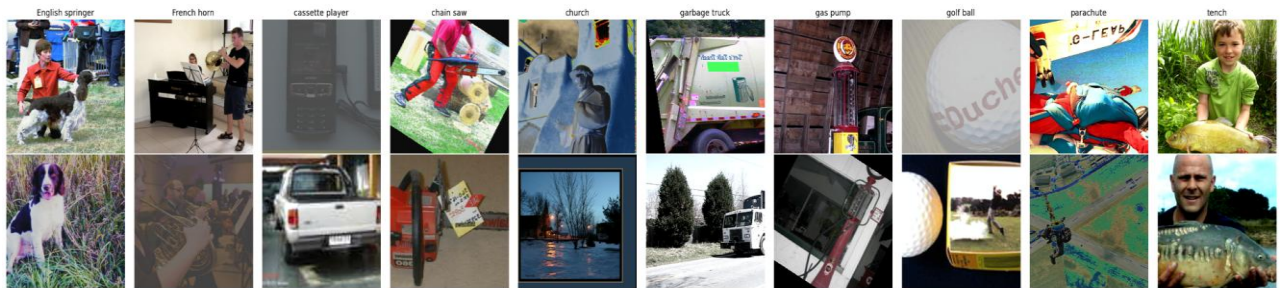


Figure 3.10. Real images of Imagenette with transformations.



Figure 3.11. Real images of Imagewoof with transformations.

As we can see the differences compared to the real images previously shown are noticeable in many pictures, as most of the data augmentation is randomly applied to a chosen percentage of the images, not to all. Although these transformations have been made, the images are still looking pretty realistic.

3.4. Classifiers trained with real images

We will train the classifiers for 300 epochs and study the results. We begin by evaluating the progression of loss functions throughout the training process, observing how effectively each model

converges. Subsequently, we will examine the evolution of classification accuracy across both the validation and training datasets, for each of the selected datasets. After establishing the overall training dynamics, we will identify and present the best classification results achieved by each model, highlighting the epoch at which the best results were obtained. Additionally, we will evaluate and report the classification accuracy for each individual class, offering insight into the specific strengths and weaknesses of our models. Finally, a comparative analysis of the training durations is included for each model across both datasets, providing an understanding of their computational efficiency.

3.5. DCGAN. Training from scratch

A **DCGAN (Deep Convolutional Generative Adversarial Networks)** is a direct extension of the GAN described above, except that it explicitly uses convolutional and convolutional-transpose layers in the discriminator and generator, respectively. The model architecture chosen to train from scratch was the one described in pytorch official site tutorial DCGAN [4] from the original Radford paper [8], it is a simple implementation of low level image generation resolution based on batch normalization. This model is an introduction to **unsupervised learning** [1] as there is no label needed but as we need to generate images of different classes, where every class is a completely different object rather than just creating images that look realistic we will create one model per class what would indirectly convert this network to a supervised learning one. If we wanted to do an unsupervised learning, for example we just wanted to generate synthetic images of dogs, we could have used the Imagewoof database and train this model with all the dogs classes together and then DCGAN would create synthetic dogs images (but as it would not distinguish from breeds, the images could look strange) or we could have used a databases where all the pictures have similar relations of aspects and object locations as CelebA database, but it is not the goal for this project. We have the code for the generator and discriminator in the Apendix A.

3.5.1. Models Structure

The structure of the DCGAN models is a simple implementation of low-level resolution image generation based on batch normalization. The size of the discriminator images and the size of the GAN generated images was initially set to 64x64 pixels. The **generator**, G , is designed to map the latent space vector (z) to data-space. Since our data are images, converting z to data-space means ultimately creating a RGB image with the same size as the training images (i.e. 3x64x64). In practice, this is accomplished through a series of strided two dimensional convolutional transpose layers, each paired with a 2D batch normalization layer and a ReLu activation. The output of the generator is fed through a Tanh function to bring it back to the input data range of $[-1,1]$. Important to highlight the presence of the batch norm functions after the conv-transpose layers, as this is a critical contribution of the DCGAN paper. These layers help with the flow of gradients during training. An image of the generator from the DCGAN paper is shown below.

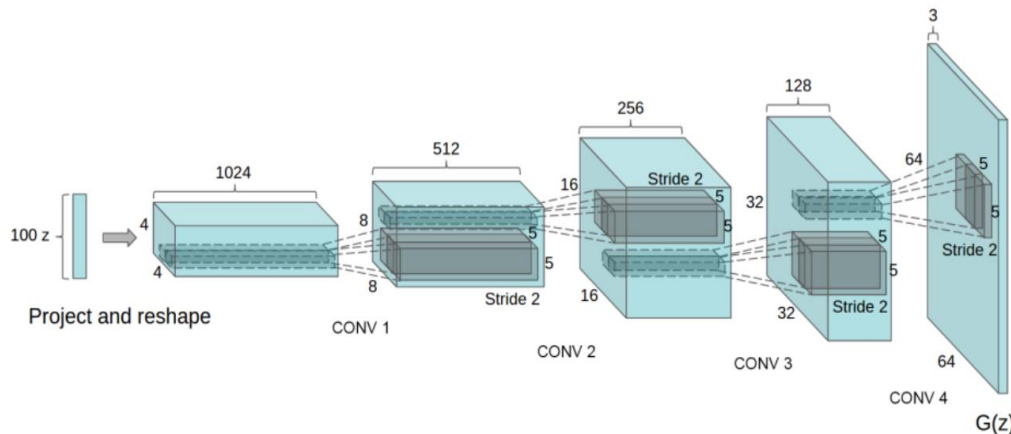


Figure 3.12. DCGAN Generator 64x64 architecture.

The **discriminator**, D , has the same structure but the other way around, if we take a look at the structure of the discriminator it is a binary classification network that takes an image as input and outputs a scalar probability that the input image is real (as opposed to fake). Here, D takes a 3x64x64 input image, processes

it through a series of Conv2d, BatchNorm2d, and LeakyReLU layers, and outputs the final probability through a Sigmoid activation function. It is a good practice to use strided convolution rather than pooling to downsample because it lets the network learn its own pooling function. Also batch norm and LeakyReLU functions promote healthy gradient flow which is critical for the learning process of both Generator and Discriminator.

3.5.2. Training Models

To be able to obtain better results each class was trained separately, having a particular discriminator and generator, this helps the model to have a better distinction of each class and not just create images that can look realistic without considering what type of object is being generated. If we only used one model for all the classes, the Generator would try to create real images of any type and the discriminator would just tell if the image is real or not without relating it to a specific class as explained in 3.5.1. For the training process we will have these variables:

- **nz** = input random noise (set to 100 in all the models). This number determines how many neurons are in the first layer and each of them would receive a random value. This random value is drawn from a normal (Gaussian) distribution with a mean of 0 and a standard deviation of 1 (standard normal distribution).
- **lrD** = learning rate discriminator
- **lrG** = learning rate generator
- **ImageSize** = pixels of the image; for example, 64 means 64x64
- **uD** = number of epochs until update the discriminator model weights (backpropagation) (set to 1 in all models). This could help to compensate if one model learns faster than the other.
- **uG** = number of epochs until update the generator model weights (backpropagation) (set to 1 in all models). Same as uD, could help to compensate if one model learns faster than the other.
- **modData** = if the images have been modified by data Augmentation or are the original ones

Although nz, uD and uG were constant values in the code were implemented to have any type of value to help the generalization of the functions

We will train the DCGANS models for 300 epochs and study the results. As the classifiers training, Initially, it will be analyzed the progression of loss functions throughout the training process, observing how effectively each model converges. Then it will be shown how the Discriminator and Generator progress in the generation of fake images showing the evolution of either the percentage of generated synthetic images incorrectly classified as real by the discriminator and the discriminator's accuracy, indicating the percentage of real images correctly classified as real. After this we will use our previously trained classifiers to evaluate which model generates the images that make the classifiers have the best accuracy for each class. Therefore, we will show the image creation progress of the chosen model for each class for each dataset Imagenette and Imagewoof. Finally, we will conduct a comparative analysis of the training durations for each model across both datasets, providing an understanding of their computational efficiency. The code is in appendix A.

3.5.3. Upscaling image Dimensions

An attempt was made to upscale the image size to 128x128 in order to create better quality image results by either adding an extra layer and duplicating the neurons per layer. The results obtained were that the training was unstable, with the G or the D learning significantly faster than the other. I also applied some techniques such as making the discriminator or the generator have two different learning rates and reducing the learning rate of the model that was learning too fast or either increment the learning rate of the model that was learning too slow and none of these techniques worked. In addition, I attempted to make the model that learns the fastest update only after a certain number of epochs, but it did not work either. This explains how

difficult it is to upscale models in deep learning since just adding more layers or more neurons do not necessarily mean the model's performance will improve.

3.5.4. DCGAN images results

The images results for the trained DCGAN show good learning progress but the results do not look realistic for most of the models. In order to justify why it was not chosen to train the classifiers with the simple DCGANs we will show a basic image example of one of the six models we will train in Section 4. There are two plots, the left one contains the Generator results, and the right one contains the real images examples. Each row will show 8 examples of random images from the DCGAN model on the left and 8 examples of real images on the right.

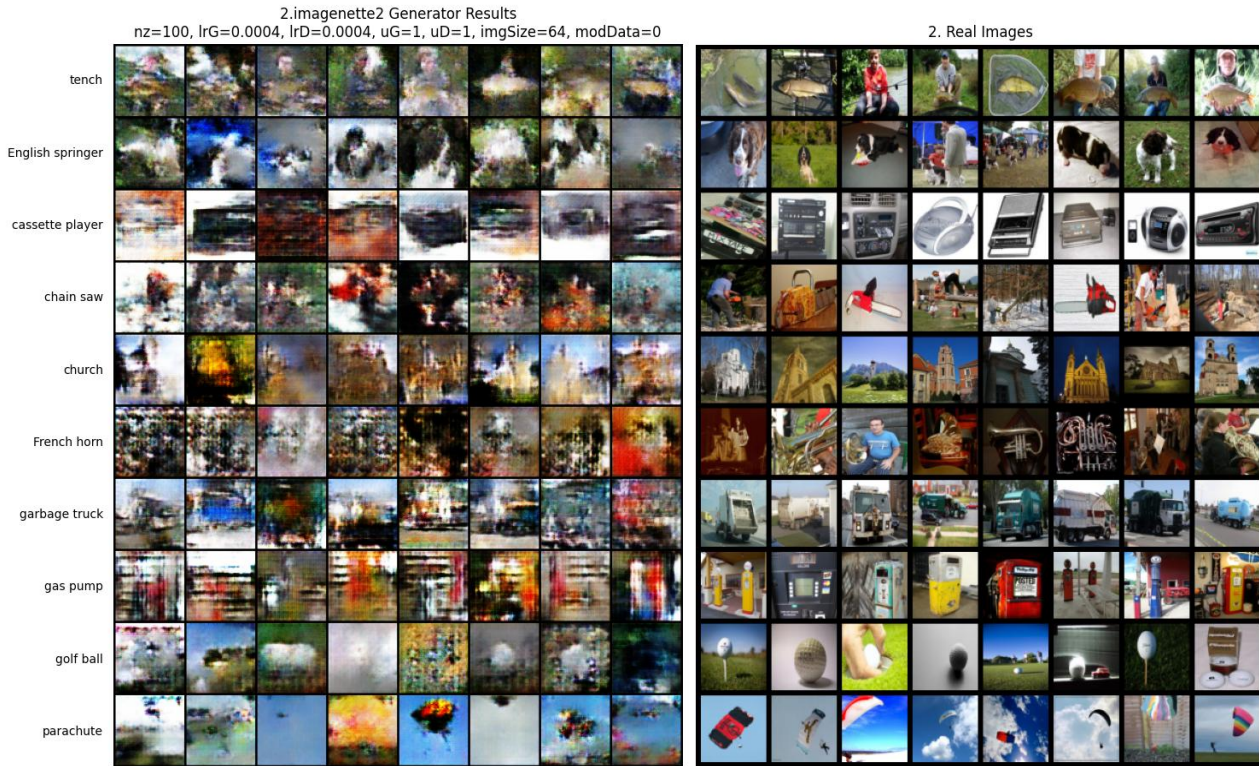


Figure 3.13. DCGAN Generator (second configuration) image creation results compared to real images.

These images show learning progress but training these DCGAN models from scratch do not generate high quality images due to the complexity of ImageNet dataset, and although they are a good simple example of how to implement a simple GAN for training the classifiers, it will be used a higher quality already trained GAN. The results of each of the six configurations are in the Appendix B.

3.6. GANs. Pretrained Models

For the creation of high-quality synthetic images it was chosen to load 5 different models with bigger internal architectures and better performance that were already trained in ImageNet that are:

- **biggan-deep-512** is a class-conditional GAN by DeepMind that uses spectral normalization, self-attention, and truncation tricks to generate high-quality images, trained on ImageNet. It generates (256x256) resolution images by default.
- **biggan-deep-512** is a deeper version of BigGAN with higher resolution (512x512), larger capacity, and improved image detail and diversity.
- **SAGAN** adds self-attention to GANs, enabling better modeling of long-range dependencies and improving image coherence and quality.
- **SNGAN** stabilizes GAN training using spectral normalization in the discriminator, leading to better image quality with simpler training.

These models are class **conditional** so the class to generate is an input variable, making it possible to generate images of the subset of Imagenette and Imagewoof. Another important thing to mention is that these

models have an input that is the **truncation**, this is made to make more variety of images by having the price to pay of losing some quality, the higher this value is the more variety of images will be created. The minimum value of truncation is 0 and the maximum level of truncation is 1. The code is in appendix A.

3.6.1. Synthetic Images Examples

Each of these GANs can generate different synthetic images for each class and due to having different structures before choosing which GAN will be used to train the classifiers some samples will be shown. We will use a low-level truncation of 0.4 to visualize some samples and choose which model generates the best images but 0.7 truncation to train the classifiers to have more variety in the synthetic image generation and help generalization.



Figure 3.14. BigGAN-deep-256 Image Creation.

a) Imagenette. b) Imagewoof.

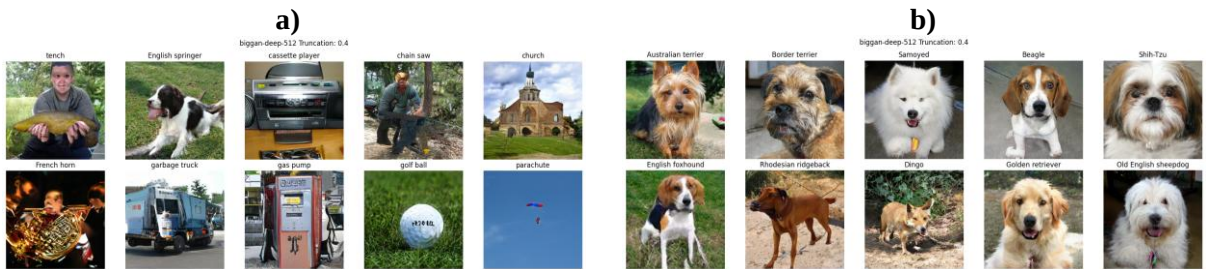


Figure 3.15. BigGAN-deep-512 Image Creation.

a) Imagenette. b) Imagewoof.

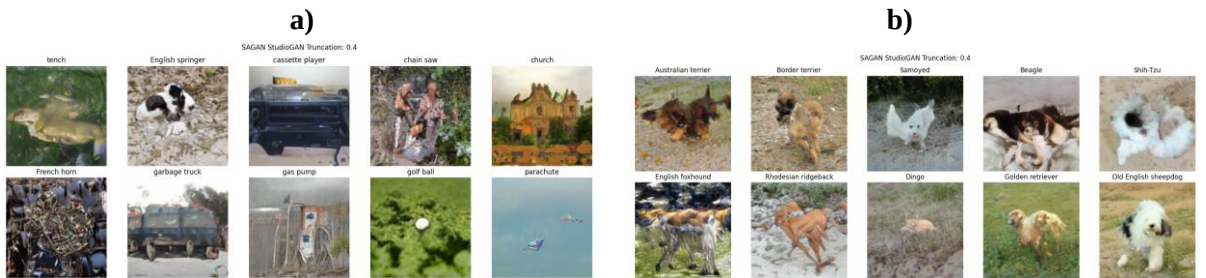


Figure 3.16. SAGAN Image Creation.

a) Imagenette. b) Imagewoof.



Figure 3.17. SNGAN Image Creation.

a) Imagenette. b) Imagewoof.

As we can see there is an enormous difference between the quality of the DCGANs models trained from scratch and these GANs.

3.6.2. Choosing the best GAN

To choose which GAN model to train between the shown ones we will use **Inception Score (IS)** and **Fréchet Inception Distance (FID)** metric which are widely used metrics for evaluating image quality generated by neural generative models, such as Generative Adversarial Networks (GANs). **Inception Score (IS)** measures both the clarity and diversity of generated images based on the outputs of a pre-trained classifier (usually Inception v3). A high IS indicates that generated images are individually clear—each image confidently represents a distinct object—and collectively diverse across various categories. However, IS does not directly compare generated images against real images, meaning it may overlook subtle discrepancies from real data distributions. **Fréchet Inception Distance (FID)** evaluates image quality by explicitly comparing the distribution of generated images to real images. It computes statistical differences (specifically, the Fréchet or Wasserstein-2 distance) between the features of real and synthetic images extracted by a pre-trained Inception v3 network. A low FID indicates that generated images closely resemble real images in content, texture, and diversity. FID typically aligns more closely with human perception of realism compared to IS, making it the current standard metric for evaluating generative image models.

Table 3.1. GAN synthetic images FID and IS.

Databases		imagenette2			imagewoof2		
Metric		fid	is_mean	is_std	fid	is_mean	is_std
1	biggan-deep-256	96.868	6.432	0.272	83.534	5.951	0.467
2	biggan-deep-512	97.258	6.722	0.428	83.902	6.138	0.116
3	SNGAN	201.747	6.034	1.034	197.535	5.685	0.549
4	SAGAN	172.534	6.458	0.505	134.439	5.956	0.66

Biggan-deep-256, the model of **BigGan** architecture that generates images of 224x224 gives the best results of the combination of fid and IS, therefore is the model chosen to generate synthetic images to train the classifiers. The internal architecture it has will be shown in Figure 3.9.

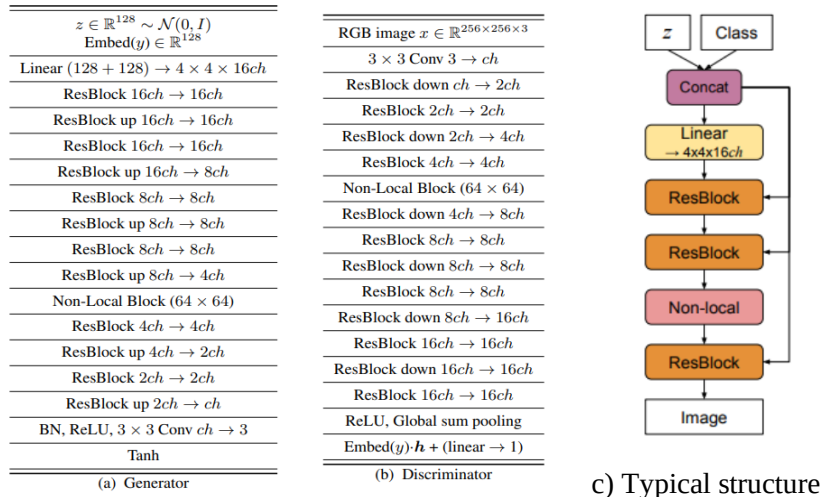


Figure 3.18. Biggan-deep-256.

This structure has been obtained from the BigGan paper *Large Scale GAN Training for High Fidelity Natural Image Synthesis* [6], by Andrew Brock, Jeff Donahue, Karen Simonyan. This is the architecture of the model, as we see it has 15 layers that follow a pattern shown in figure c). As we can see the model is very deep making it able to create sophisticated patterns

3.6.3. Is BigGan the best GAN model?

As we are using an already trained GAN model it is a good question to think if BigGan is the best possible model for our purpose. The answer is no, there is a rank of the best GAN models for ImageNet [9].

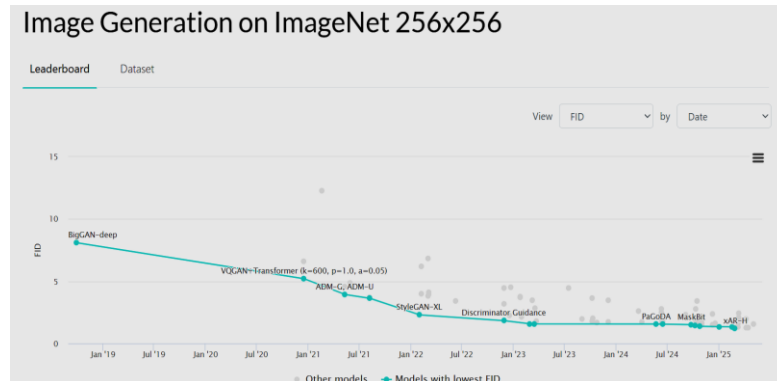


Figure 3.19. ImageNet 256x256 images FID models rank.

In this graph, BigGan is far from being the best model, although it provides good results. Then maybe the next question could be, why BigGan and not some other GAN that generates more accurate results and there are 2 main reasons. Firstly, training models with the GPU require cuda and pytorch versions compatibility and it is common to find pretrained models in different versions of torch and cuda. This leads to compilation errors due to incompatible newer versions. It tried to load StyleGAN XL from [10] which had better image quality as we can see in Figure 3.10, and it was successfully loaded in python and worked perfectly with the CPU. The problem arises when trying to load the model to the GPU because due to having pytorch version of 1.9.1 instead of 2.0.1 and cuda version 11.1 instead of 11.8, it did not compile. If we explore the possibility of making the calculations in CPU, for creating one batch of one image of each class it took 1 minute and 40 seconds which makes it extremely slow and inefficient to train the classifiers. As the rest of the code was in other pytorch and cuda version it was decided to use BigGAN. Then on the other hand, the better the synthetic images are, the smaller the difference it will encounter to the models trained with real images, so it is interesting to not use the best model to see how the classifiers work with noticeable fake images having this realistic quality gap between the synthetic and real data.

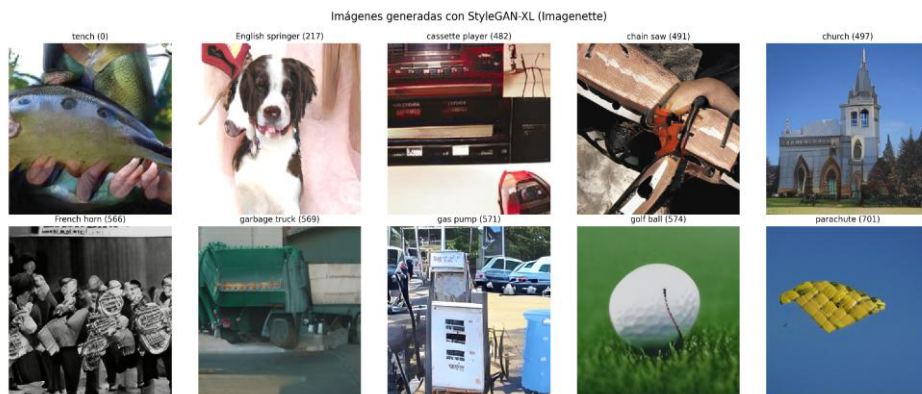


Figure 3.20. StyleGAN-XL image creation for ImageWoof with truncation of 0.4.

As we can see, the images generated are like the BigGan model ones.

3.7. Classifiers trained with fake Images

The second part of this project is to train the five different chosen classifiers with synthetic generated by **BigGan** and compare the performance on classifying real images that have never been seen before. The same training process for the classifiers trained with real images will be applied but changing the real images to the synthetic ones, generated by the BigGan model. Then we will compare the performance of these classifiers to the ones trained with real images.

Results

The following section presents and analyzes the results obtained from the experimental evaluations carried out throughout this work. Initially, we explore the training performance by monitoring key metrics such as loss progression and accuracy for classifiers and GAN models. Subsequently, we highlight the best-performing models and show the results obtained for all the models. Finally, we compare and discuss the computational resources used by each model, particularly focusing on their training times.

4.1. Classifiers trained with real images

The classifiers were trained for 300 epochs separately for the Imagenette and Imagewoof database.

4.1.1. Loss evolution

First, we will take a look at our **loss functions progression** during the training in Figure 4.1.

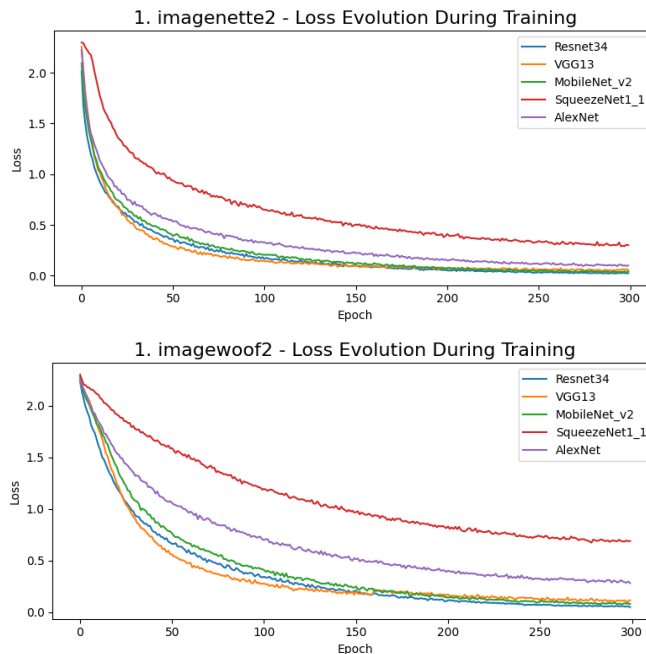


Figure 21. Classifiers trained with real images loss.

The progression of the loss is working pretty well as we can see a big fall in the first epochs as the model is learning the most and then once the model knows the basics, it slowly converges to 0. Also, we do not observe big gaps or jumps upwards or downwards which tells us that the training is very stable. A difference that is worth highlighting between the Imagenette and Imagewoof graph is that for the imagewoof dataset, all the models learn slower due to the similarity between the classes, making the first epochs to not have the beginning loss drop as big as the first epochs.

4.1.2. Accuracy progression

Then we can check out **accuracy progression** on our validation and training datasets for both datasets.

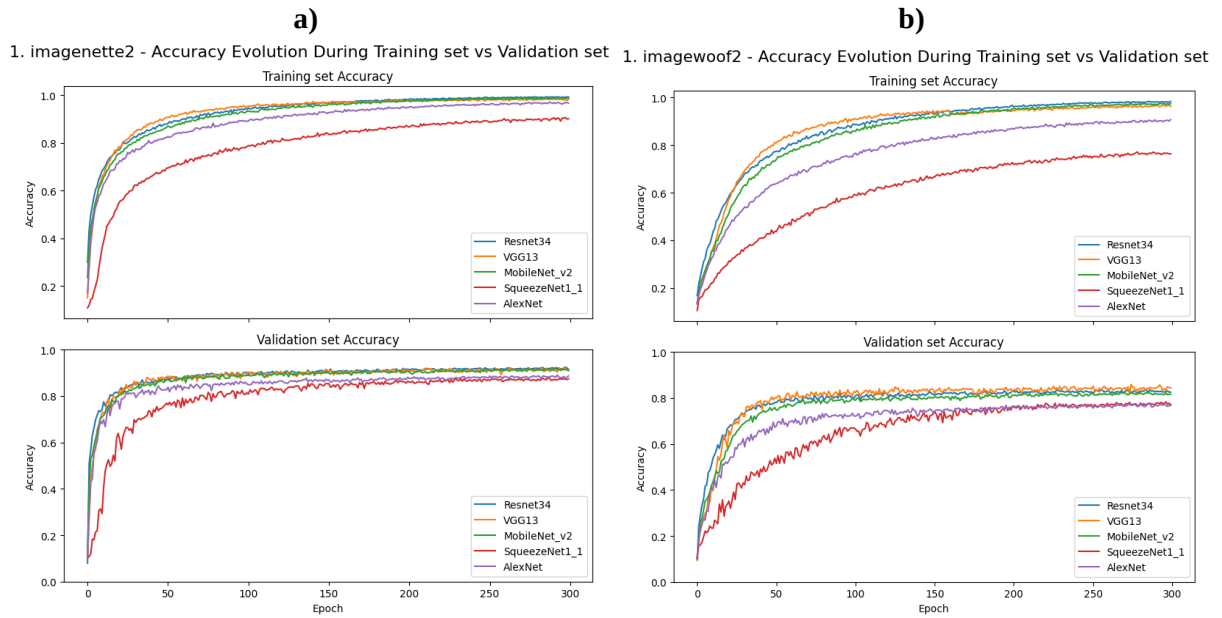


Figure 22. Classifiers trained with real images accuracy progression.

a) Imagenette training and validation dataset, b) Imagewoof training and validation dataset

Due to the data augmentation applied techniques we can see that although the training dataset obtains a high accuracy very early, the validation accuracy keeps growing as the model becomes able to generalize what is learning, leading us to have better results when we train for enough epochs. We can also see that the model learns very fast in just around 50 epochs in imagenette and around 75 in imagewoof by then we can assume that the model has learned the basic patterns of each class. Afterwards, it slowly gets better by learning harder patterns.

4.1.3. Best results

Now we will look at the **best results obtained for each model** and at what epoch they were reached. It is very important to understand this table that the accuracy of the results was done using the model version at the shown epoch, where the validation results were the highest.

Table 4.1. Classifiers trained with real images accuracy.

Databases	imagenette2					imagewoof2				
	Top 1		Top 5		Epoch	Top 1		Top 5		Epoch
	train-1	val-1	train-5	val-5		train-1	val-1	train-5	val-5	
AlexNet	0,977	0,887	0,999	0,985	293	0,918	0,778	0,993	0,973	282
MobileNet_v2	0,99	0,919	<u>1</u>	0,992	292	0,98	0,838	<u>1</u>	0,979	275
Resnet34	0,994	<u>0,922</u>	<u>1</u>	<u>0,994</u>	298	<u>0,984</u>	0,837	<u>1</u>	0,979	256
SqueezeNet1_1	0,899	0,87	0,993	0,987	288	0,766	0,775	0,972	0,973	297
VGG13	<u>0,986</u>	<u>0,922</u>	<u>1</u>	0,993	297	0,963	<u>0,849</u>	0,999	<u>0,985</u>	275

The code for the calculation is in appendix A.

The best model for Imagenette is **Resnet34** and **VGG13** having an accuracy draw of **92,2%** and for Imagewoof is **VGG13** having an accuracy of **84,5%**. The best result hit was at epochs really close to 300, for ResNet 298, which can tell us that the model was not suffering from overfitting yet and training for some epochs could get slightly higher scores.

Table 4.1. just shows us the overall accuracy results for each model, but we do not distinguish from which classes are having better classifications results and which one worse result. In Figure 4.3. will be shown the **classification accuracy per class** for **Imagenette** and in Figure 4.4. for **Imagewoof**.

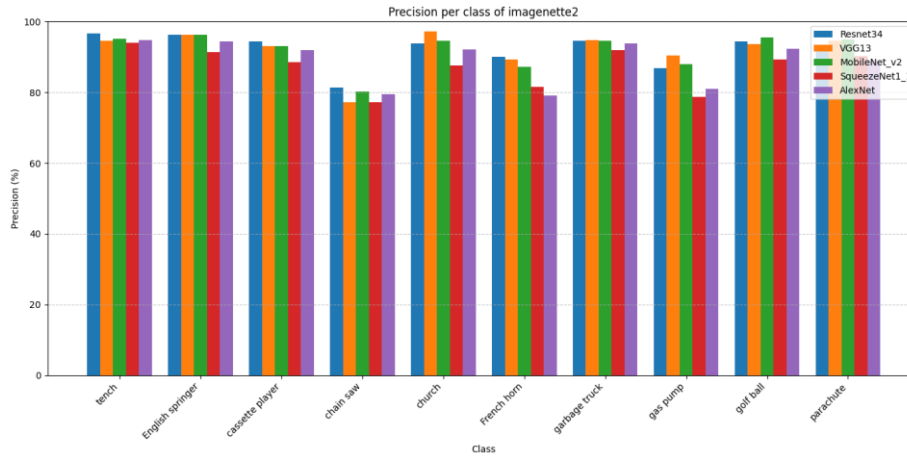


Figure 23. Accuracy per class of classifiers trained with synthetic images Imagenette.

High classification accuracy is achieved for most classes, especially **English springer**, **cassette player**, **chain saw**, **church**, and **golf ball**, where accuracy generally exceeds or closely approaches 90%. Slightly lower accuracies appear for the classes **gas pump** and **garbage truck**, although these still show reasonable performance, typically above 80%. Performance across models is relatively consistent, with only minor variations observed.

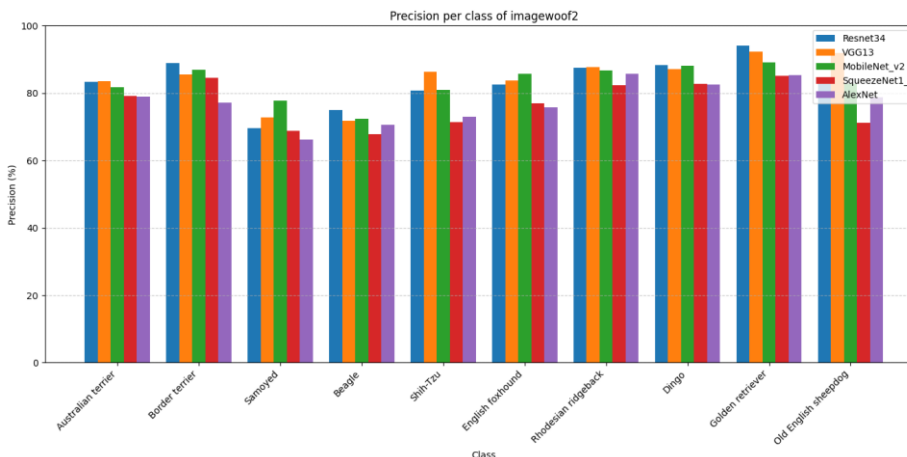


Figure 24. Accuracy per class of classifiers trained with synthetic images Imagewoof.

The highest accuracies are seen for the classes **Rhodesian ridgeback**, **English foxhound**, and **Australian terrier**, with classification accuracy frequently around or above 80%. Classes such as **Beagle**, **Golden retriever**, **Shih-Tzu**, and **Border terrier** exhibit lower classification performance, usually within the 70% accuracy range. The variations between models are more pronounced, particularly noticeable for classes with lower accuracy, suggesting different model sensitivities to certain visual features.

4.1.3. Training time

Finally, we will compare the **training time** of each model in each dataset in Figure 4.5.

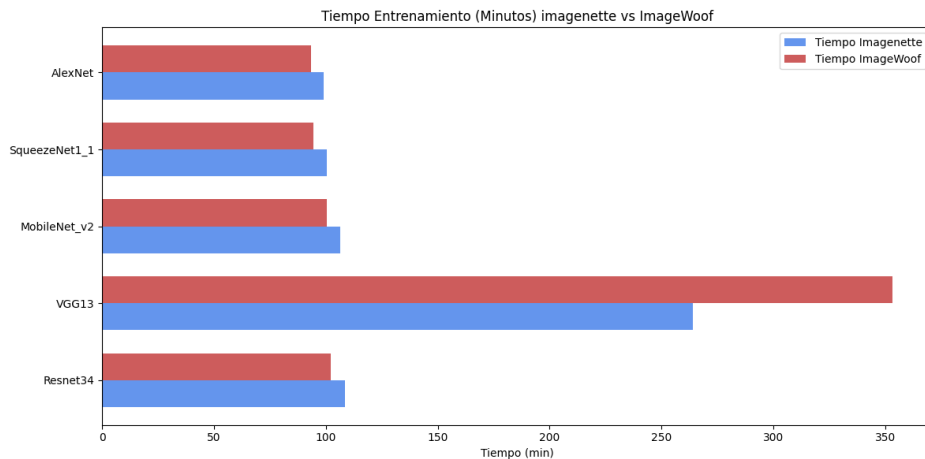


Figure 25. Training time for Imagewoof and Imagenette, classifiers trained with synthetic images.

Although the result might look strange, having a model that takes almost the same time as the other 4 models, there is an explanation. It was tried to see how just a small difference in loading more data than the amount that fits on the VRAM affected. During the VGG13 training, the amount of total VRAM used was a bit higher than the max VRAM, making the previously mentioned page fault. This mixed with its biggest architecture caused the model to be by far the one that took the longest. In order to choose the best model based on balancing accuracy and efficiency, MobileNet would be a very good approach to combine a low cost of training with accurate results.

4.2. DCGAN. Training from scratch

Now it will be shown how a DCGAN is trained from scratch to be able to generate synthetic images.

4.2.1. DCGANs training configurations chosen

Due to the sensibility of the learning rate in GANs models, the learning rate can make the generator or the discriminator learn faster and surpass the other making the training ineffective [3]. Another key concept is that these GANs are usually trained with enormous amounts of data and in Imagenette and Imagewoof there are around one thousand images per class which may not be enough, so it will be tried to apply data augmentation techniques to evaluate different approaches. The chosen models are:

1. **lrD=0.0002, lrG=0.0002, modData=0**
2. **lrD=0.0004, lrG=0.0004, modData=0**
3. **lrD=0.0006, lrG=0.0006, modData=0**
4. **lrD=0.0002, lrG=0.00026, modData=1**
5. **lrD=0.0004, lrG=0.00052, modData=1**
6. **lrD=0.0006, lrG=0.00078, modData=1**

The first 3 models will just have a bigger learning rate in both generator and discriminator. The last 3 will have 2 modifications, first the generator will have a 30% bigger learning rate and we will apply the data augmentation techniques to the training set.

4.2.2. Loss evolution

In this section we will show the loss per model of the two databases, Imagenette and Imagenet. There will be 6 groups of 2 plots for each configuration to distinguish better the discriminator and generator loss values. Inside each plot we will have the values for the 10 different classes we are studying. The x value will represent the epochs and the y value the loss value.

First for **Imagenette** in **Figure 4.6**.

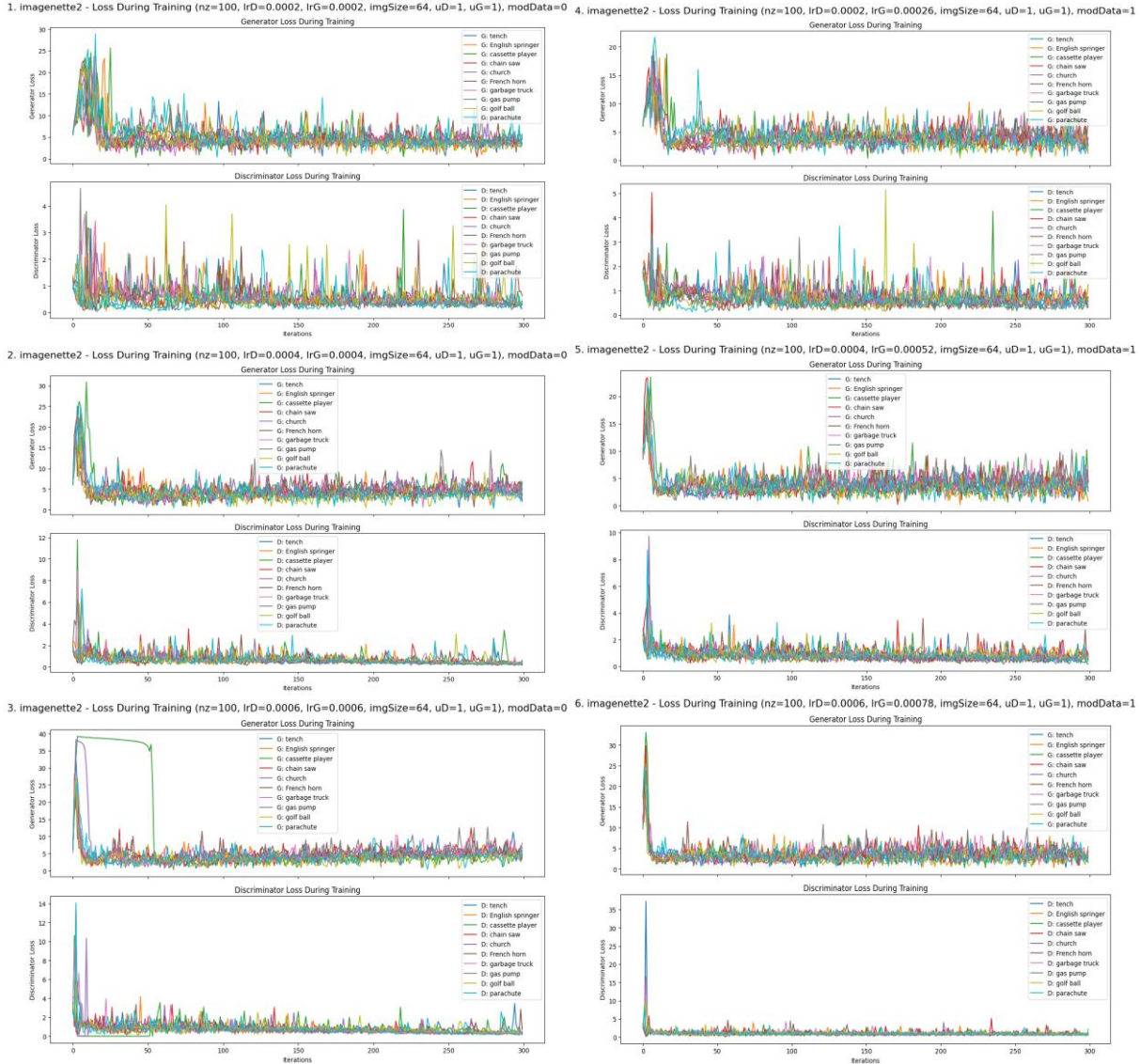


Figure 26. DCGAN loss evolution during training Imagenette.

1-3) First 3 configurations, Increasing the learning rate from 0.2 to 0.6 without using Data Augmentation techniques.
4-6) Last 3 configurations, Increasing the learning rate from 0.2 to 0.6 in discriminator and adding a 30% increase of the learning rate in the generator, from 0.26 to 0.78 and using DataAugmentation techniques

The first thing to mention is that the loss values directly depend on the class we are training as in some graphs we can have some models learning progress that are more stable than others. Secondly as the learning rate for the discriminator was increased the more stable it became, although we can see that the greater the discriminator update was the more difficult it made the generator converge to 0 as the discriminator learned faster. With data augmentation techniques, the training resulted in more stability as the discriminator had a bigger challenge as now there were more versions of the images in the dataset.

Now for **Imagewoof** in Figure 4.7.

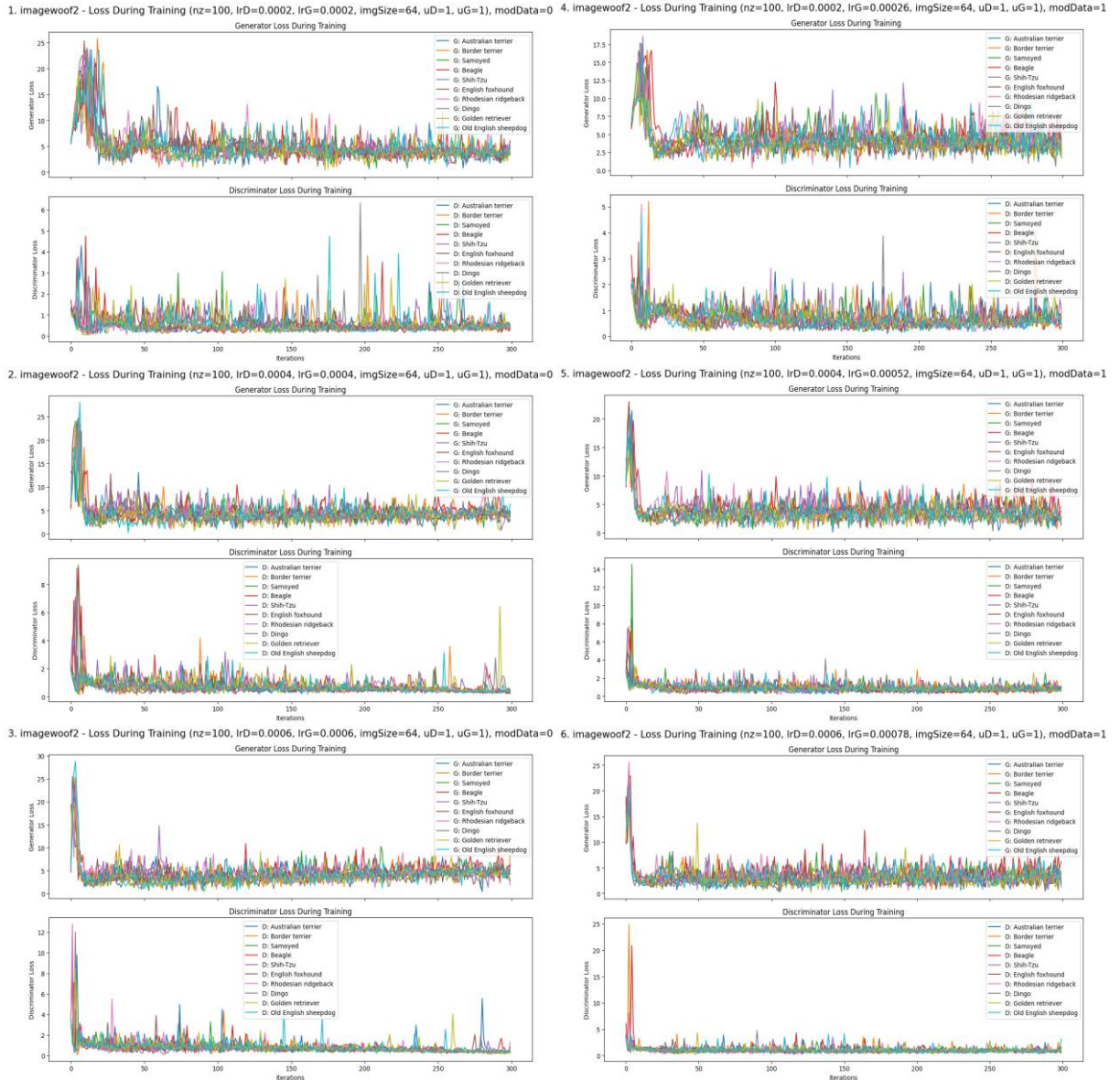


Figure 27. DCGAN loss evolution during training Imagewoof.

1-3) First 3 configurations, Increasing the learning rate from 0.2 to 0.6 without using Data Augmentation techniques.
 4-6) Last 3 configurations, Increasing the learning rate from 0.2 to 0.6 in discriminator and adding a 30% increase of the learning rate in the generator, from 0.26 to 0.78 and using DataAugmentation techniques

The results obtained are similar to Imagenette. There is no big change appreciated in the plots, do not see anything extra worth mentioning.

4.2.3. Accuracy progression during training

In this section we will show the accuracy per model of the two databases, Imagenette and Imagewoof. There will be 6 groups of 2 plots for each configuration to distinguish better the discriminator and generator accuracy values. The top plot is the generator one and shows the % of fake images classified as real and the bottom one shows the % of real images classified as real. Inside each plot we will have the values for the 10 different classes we are studying. The x value will represent the epochs and the y value the proportion of accuracy in a scale of 0-1.

First for Imagenette

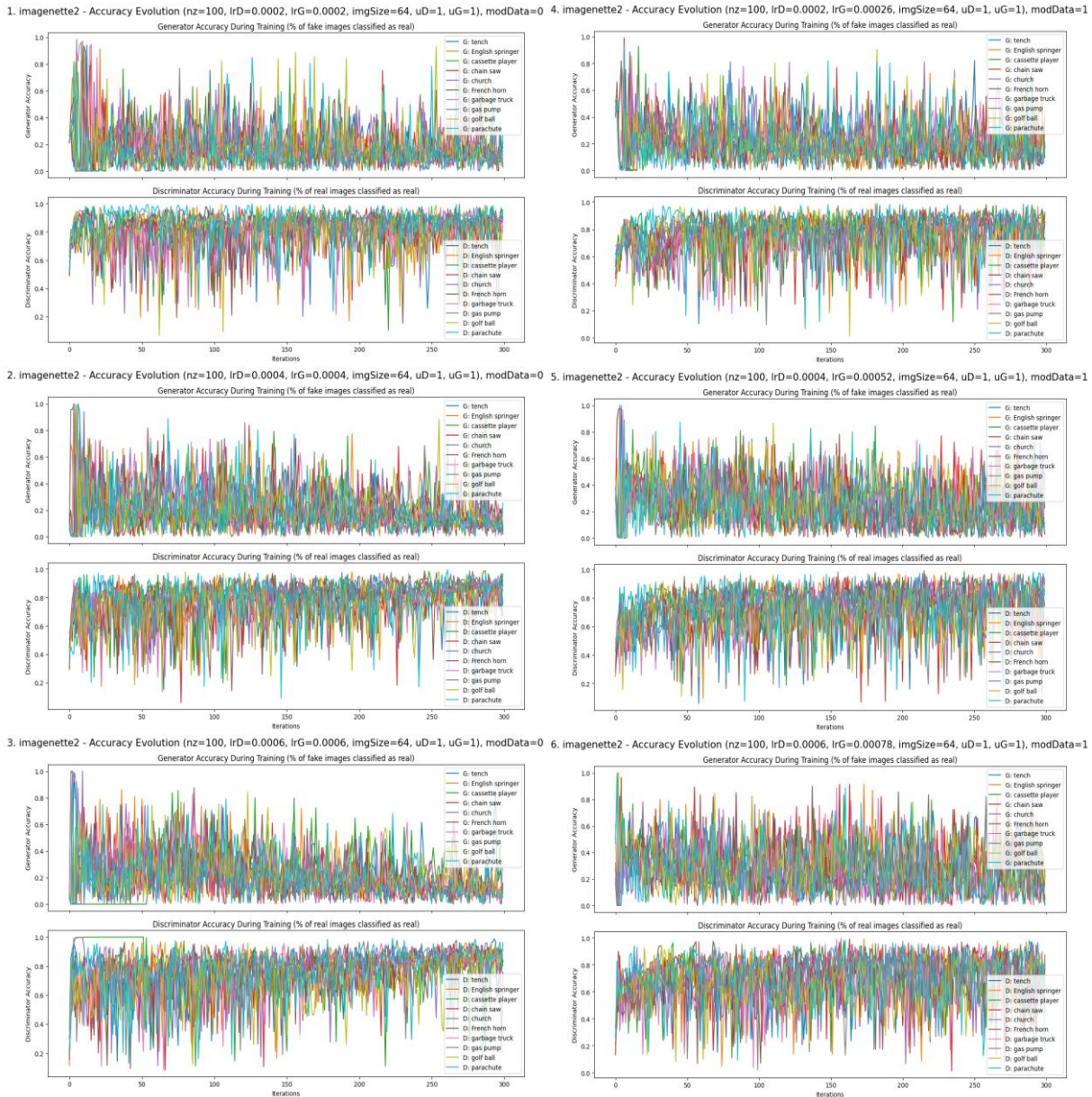


Figure 28. DCGAN accuracy evolution during training Imagenette.

1-3) Increasing the learning rate from 0.2 to 0.6 without using Data Augmentation techniques. 4-6) Increasing the learning rate from 0.2 to 0.6 in discriminator and adding a 30% increase of the learning rate in the generator, from 0.26 to 0.78 and using DataAugmentation techniques

The first thing worth mentioning is that as we saw in the loss evolution graphs, for bigger learning rates for the 1-3) figures in Figure 4.8, the discriminator clearly wins the generator, classifying almost pretty good the real and fake images. With the change in the learning rates and data augmentation of the graphs 4-6) seems to be a better balance although it does not necessarily mean that the image creation is better than the others. In these graphs from 4-6) the tendency of the discriminator to classify the real images seems to be a slower tendency to 1.

Then for Imagewoof

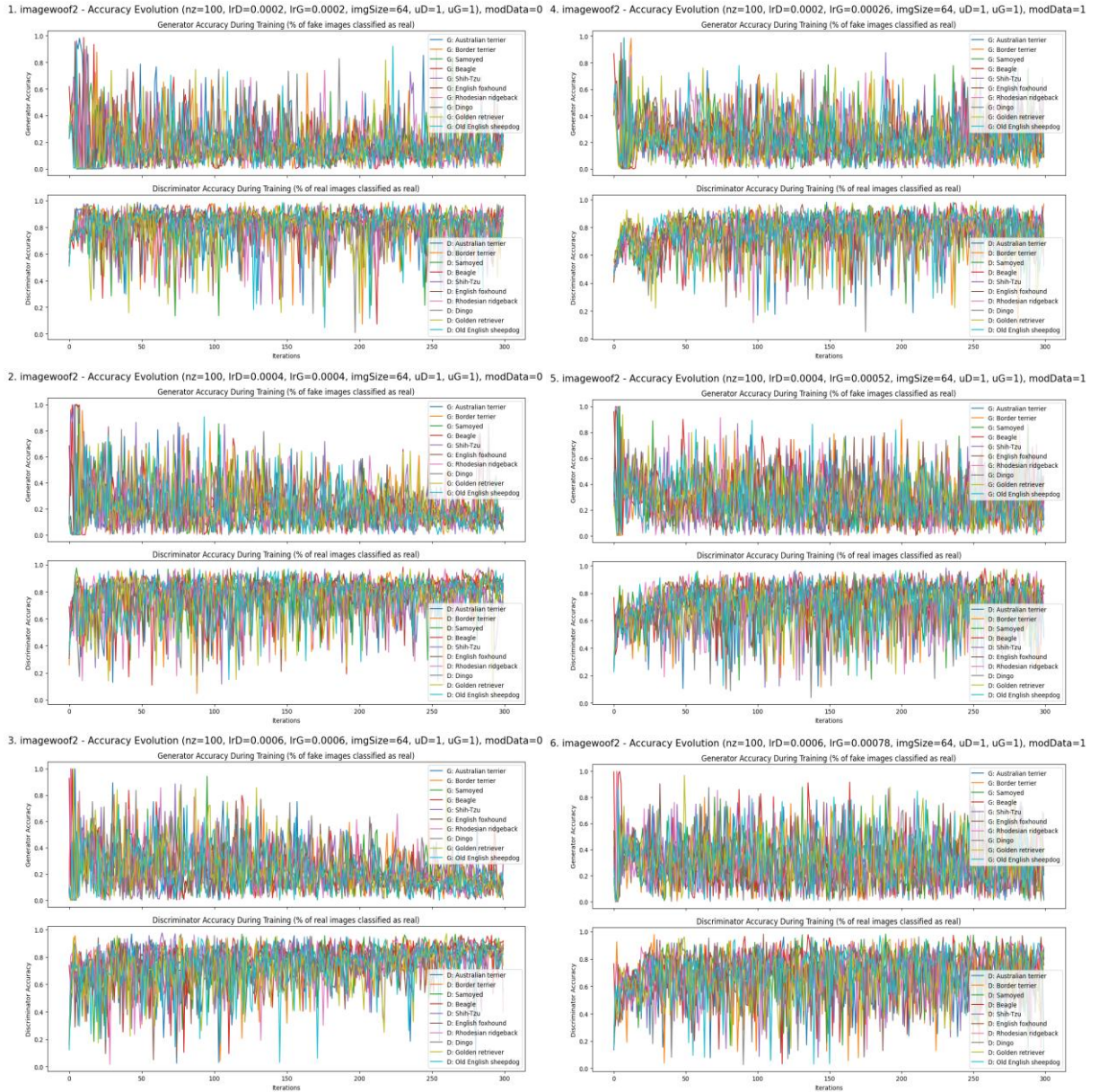


Figure 29. DCGAN accuracy evolution during training Imagewoof.

1-3) Increasing the learning rate from 0.2 to 0.6 without using Data Augmentation techniques. 4-6) Increasing the learning rate from 0.2 to 0.6 in discriminator and adding a 30% increase of the learning rate in the generator, from 0.26 to 0.78 and using DataAugmentation techniques

Again, the results look similar, which makes sense because the models are trained individually for each class and they do not know how to generate any other image that is not the one they were trained for, so the similarities between classes do not affect. Nothing else noticed that looked worth mentioning.

4.2.4. Time trained

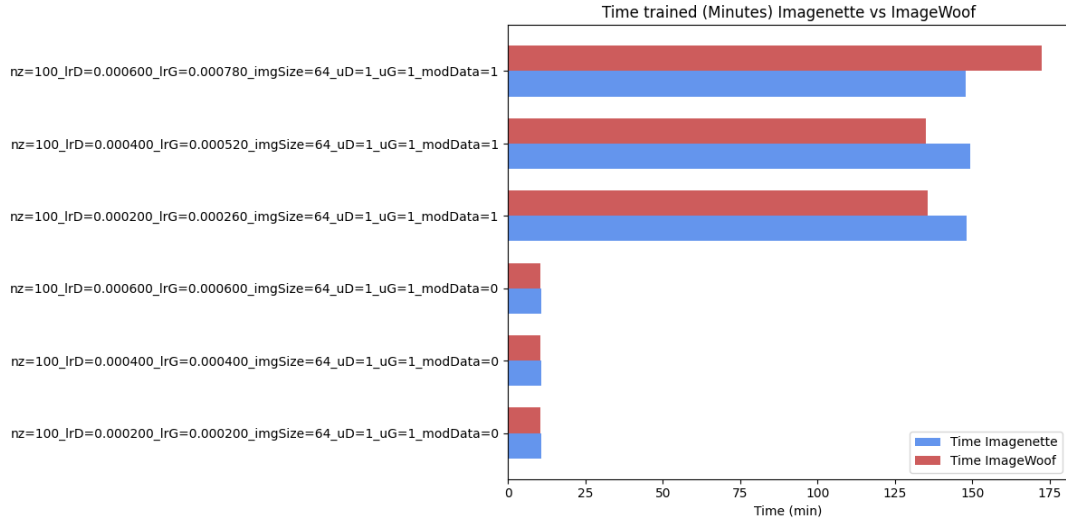


Figure 30. DCGANs time trained.

As we can see the 3 first models have been trained with modified data, which made them have to transform the data in each epoch. For the other models, as there was not any dynamic data Augmentation technique, a preload function was applied to transform all the images to pytorch tensors to afterwards accelerate the training time (both codes are in the Appendix A). This factor of training time is considerable as Data Augmentation will help to make the models generalize better and prevent memorization of previous parameters, but it has the price to pay of longer training time and consequently more resources consuming. The lowest time is 10 minutes and the longest 170 minutes, the difference is a factor of ≈ 17 , which is an enormous difference.

4.2.5. DCGANs best model

The way of deciding which DCGAN provides the best results in this project is to use the best of the already trained (with real images) Classifiers to evaluate the generation of the fake images. We could have used other techniques such as FID or IS but as we are using those techniques to evaluate the preloaded GANs we will see afterwards, it has been chosen to use a different method to examine another possibility. As for each configuration we are training one model per class (10 models for configurations). The classification ability to distinguish the classes of every model will be tested to obtain the best model configuration for each database class. These are the results, the marked yellow results are the highest achieved in each class.

For **Imagenette** are shown in Table 4.2. and for **Imagewoof** in Table 4.3.

Table 4.2. Best Classifier accuracy with DCGAN images Imagenette.

Model Config	tench	English springer	cassette player	chain saw	church	French horn	garbage truck	gas pump	golf ball	parachute
1	0.02	0.18	0.20	0.44	0.02	0.04	0.03	0.00	0.95	0.31
2	0.03	0.15	0.24	0.47	0.03	0.01	0.04	0.01	0.84	0.53
3	0.05	0.09	0.19	0.64	0.08	0.01	0.07	0.00	0.76	0.34
4	0.10	0.11	0.33	0.14	0.01	0.00	0.02	0.00	0.93	0.41
5	0.03	0.17	0.13	0.40	0.00	0.01	0.03	0.00	0.87	0.37
6	0.00	0.14	0.40	0.06	0.00	0.00	0.01	0.00	0.88	0.38

The best classification accuracy is obtained for the class **golf ball**, achieving notably high values (up to **0.95** accuracy). The class **chain saw** and **parachute** also show relatively good results (having **0.64** and **0.53** respectively). On the other hand, classes such as **tench**, **French horn**, **garbage truck**, and **gas pump**

demonstrate poor results, with accuracies close to 0, suggesting these classes are challenging for models trained on synthetic images.

Table 4.3. Best Classifier accuracy with DCGAN images Imagewoof.

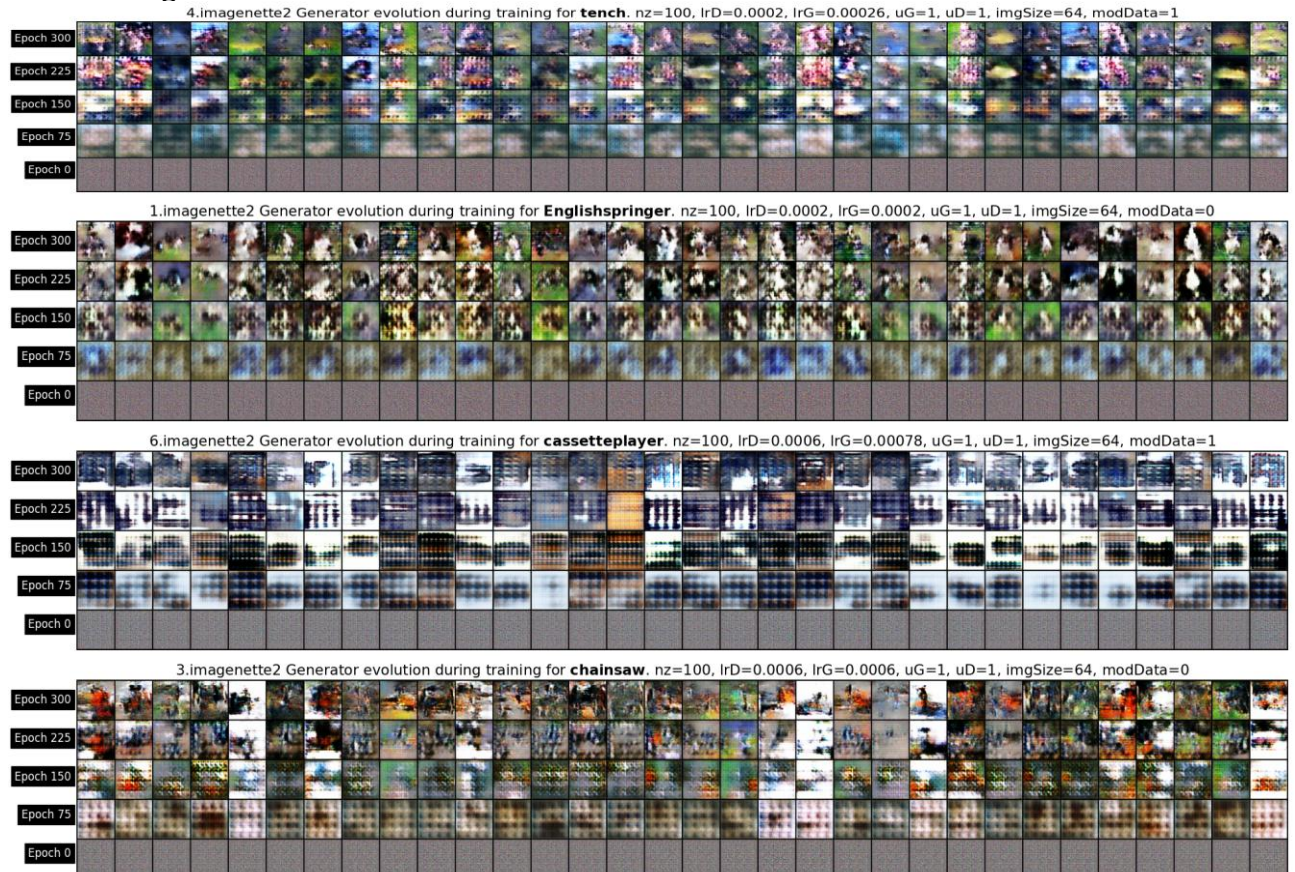
Model Config	tench	English springer	cassette player	chain saw	church	French horn	garbage truck	gas pump	golf ball	parachute
1	0.25	0.01	0.13	0.04	0.05	0.57	0.27	0.02	0.28	0.24
2	0.43	0.01	0.07	0.18	0.06	0.62	0.53	0.11	0.26	0.38
3	0.36	0.00	0.12	0.20	0.17	0.58	0.34	0.10	0.26	0.25
4	0.18	0.03	0.02	0.00	0.01	0.35	0.49	0.02	0.41	0.03
5	0.46	0.00	0.03	0.02	0.06	0.56	0.24	0.05	0.21	0.09
6	0.50	0.00	0.04	0.10	0.09	0.34	0.26	0.16	0.14	0.41

The classes **English foxhound**, **Australian terrier**, and **Rhodesian ridgeback** achieve relatively better results, with accuracies up to **0.62**, **0.50**, and **0.53**, respectively. Conversely, expect the **Golden retriever** and **Old English sheepdog** that gets **0.41** the rest of the models have a performance minor or equal to **0.2** which are pretty low results. The code for the calculation is in appendix A.

4.2.6. Showing best DCGANs image creation process

Let's see how each model learns in every epoch to form input noise generate synthetic images. As we trained for 300 epochs, we will visualize the progress for each class after 75 epochs, so at epoch 0, epoch 75, epoch 150, epoch 225 and epoch 300. The 6 different model configurations will be shown with a number at the beginning (being 1 the first configuration and 6 the last one) of each plot being the first configuration 1 and the last configuration 6. There will be one plot for each class, showing the model that got the most accuracy in Tables 4.2 and 4.3 Best Classifier accuracy.

First for **Imagenette**



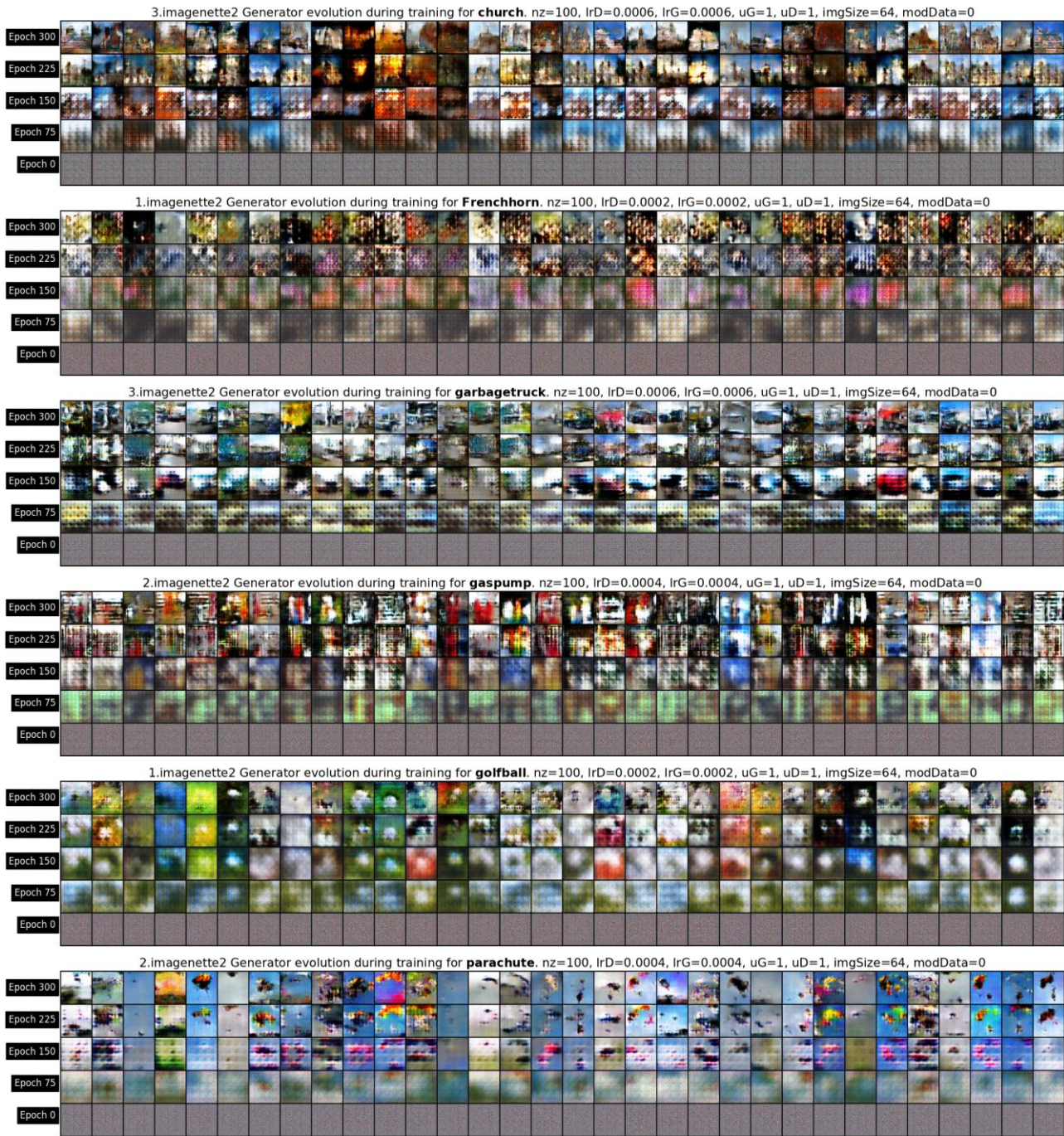
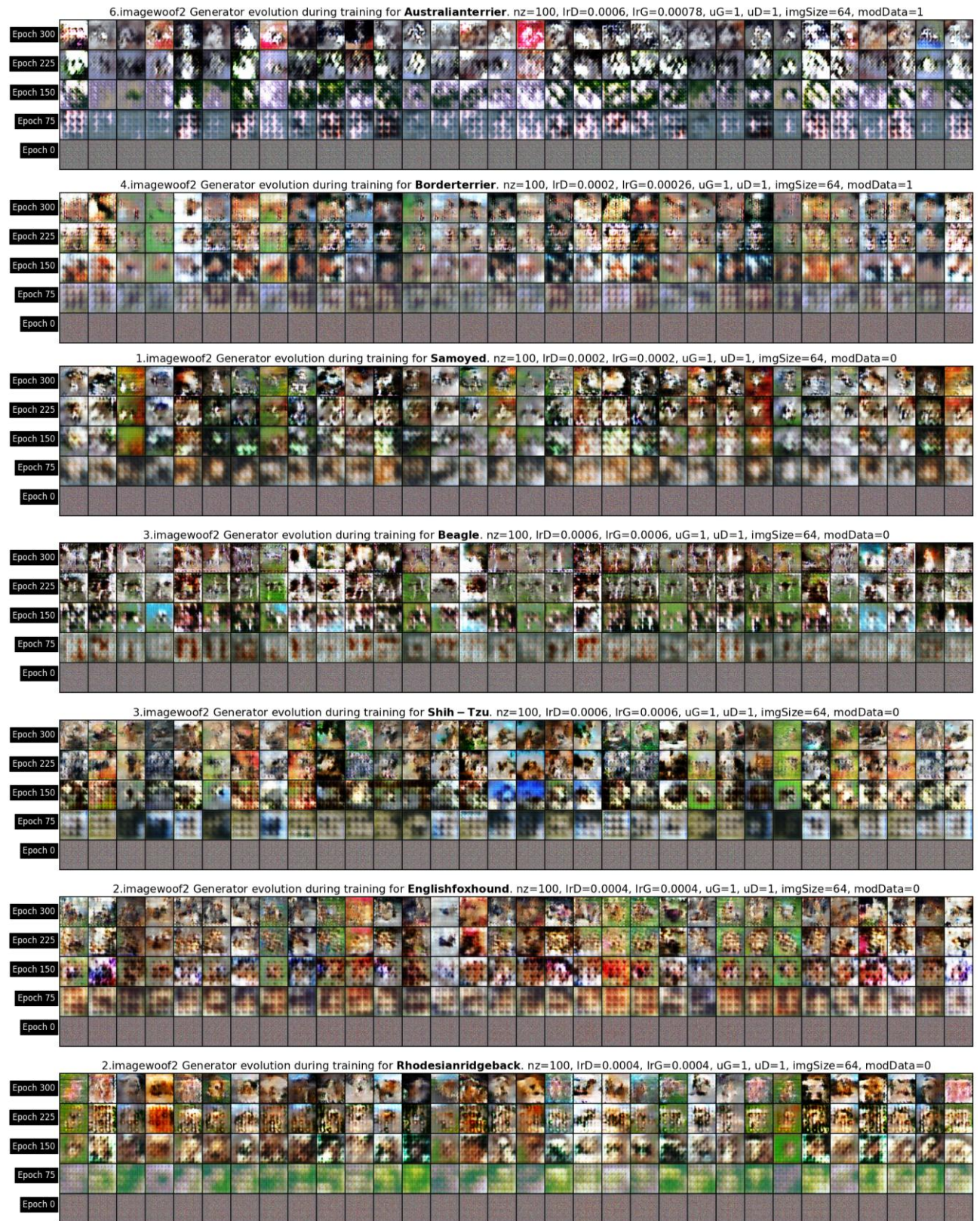


Figure 31. DCGAN synthetic image creation evolution for Imagenette.

Shown for each class, with models trained for 300 showing the evolution of the image creation after 75 epochs

During the learning process, we can see that the models, trained without data augmentation techniques, easily get the image of typical colors first and from there starts creating shapes. Then to improve the shapes created and generate the realistic structures of some classes it struggles. It can be stated that there is learning progress on the images that the DCGAN generates making them look more realistic every 75 epochs.

Now for for **Imagewoof**



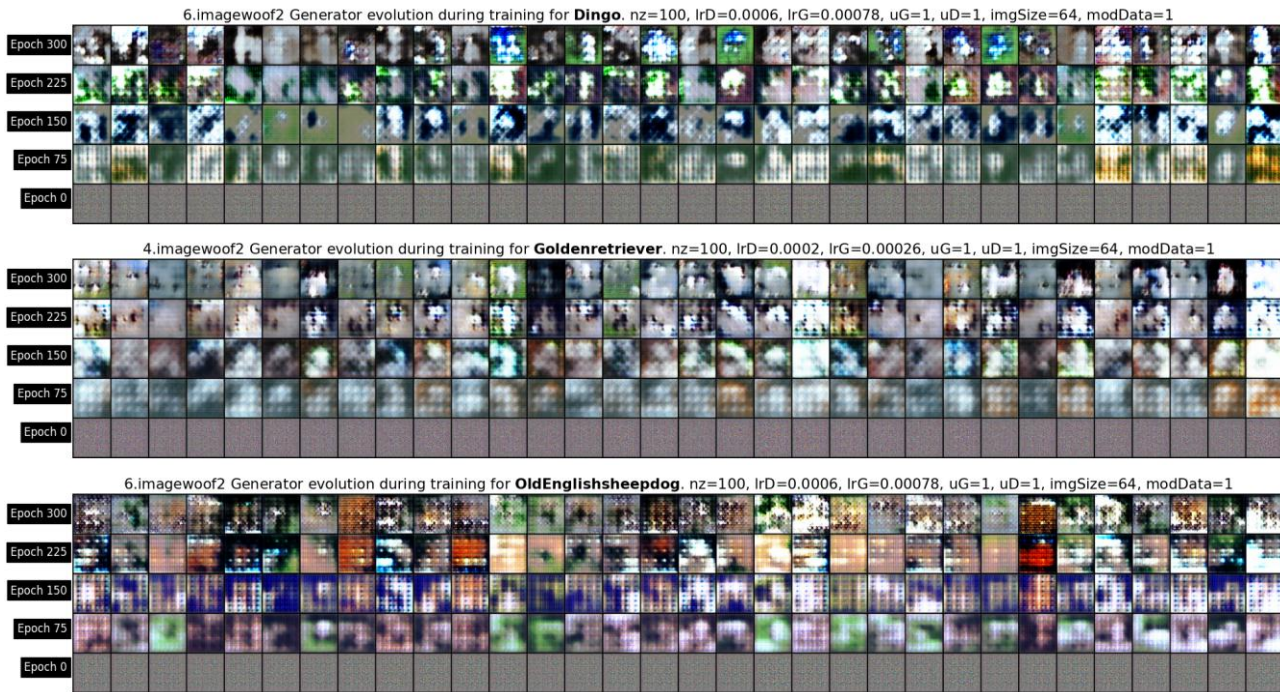


Figure 32. DCGAN synthetic image creation evolution for Imagewoof.

Shown for each class, with models trained for 300 showing the evolution of the image creation after 75 epochs

Although this progression is notable, the difficulty of the ImageNet real-world variability super pass these simple DCGAN models, maybe if we were using a simple database these DCGAN models would have been enough. The final images of these DCGANs do not have enough quality in most of the models, they were shown in Figure 4.12. In order to have more realistic results it was chosen to use a GAN already trained in ImageNet that makes more realistic images.

4.3. Classifiers trained with fake Images

The second part of this project is to train the chosen classifiers with synthetic images and compare the performance to the ones trained with real images. First, we will start visualizing how the images the BigGan generates look after the Data Augmentation techniques. As specified before the synthetic

4.3.1. Data augmentation in synthetic images

First, we will take a look, in Figure 4.13. and 4.14., how the **data augmentation** techniques that were explained in section 3, affect to the variety of images created during the training.



Figure 33. GAN synthetic images of Imagenette with transformations.



Figure 34. GAN synthetic images of Imagewoof with transformations.

As we can see the differences compared to the real images previously shown are noticeable in many aspects. These data augmentation techniques will help the model generalize better as it would be less dependent on classifying by colors or object locations, and it will be able to learn harder patterns or shapes. Also, the truncation of the model was set to 0.7 to make a great variety of images but conserving the quality. This truncation provides us with a variety of images but for some models such as tench we can see that the pattern of images is the same in all examples and in other classes such as border terrier or French horn, the quality of the images does not look real.

Now the results of training the same 5 classifiers as for the real images for 300 epochs will be shown.

4.3.2. Loss evolution

First, in Figure 4.15 are shown our **loss functions evolution** during the training.

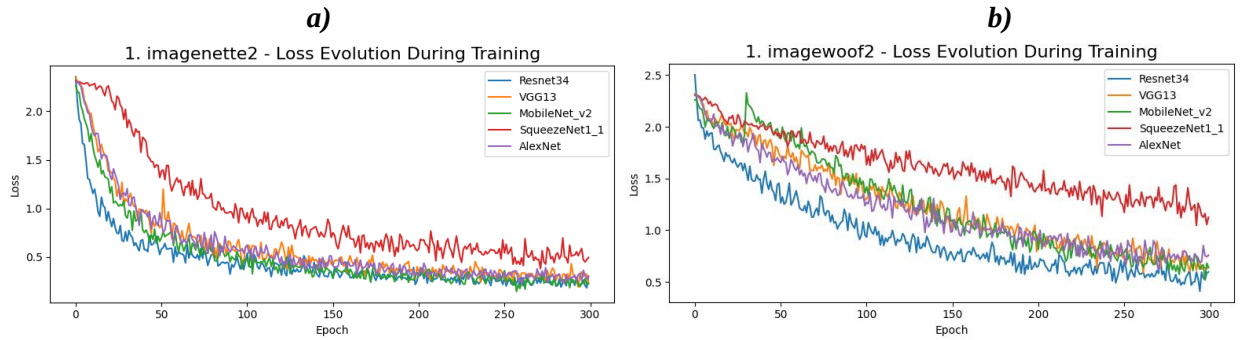


Figure 35. Classifiers trained with synthetic images loss.

a) Imagenette loss b) Imagewoof loss

As for the real images, the model learns faster with the imagenet dataset than the imagewoof dataset due to the difference of difficulty the databases have. We can notice that for the imagewoof database the loss does not converge to 0 that easily and the lowest value achieved is around 0.5. Both models' loss function progression is normal; the loss keeps reducing every epoch.

4.3.3. Accuracy evolution

Then we can check out **accuracy evolution** on our validation and training datasets for both datasets.

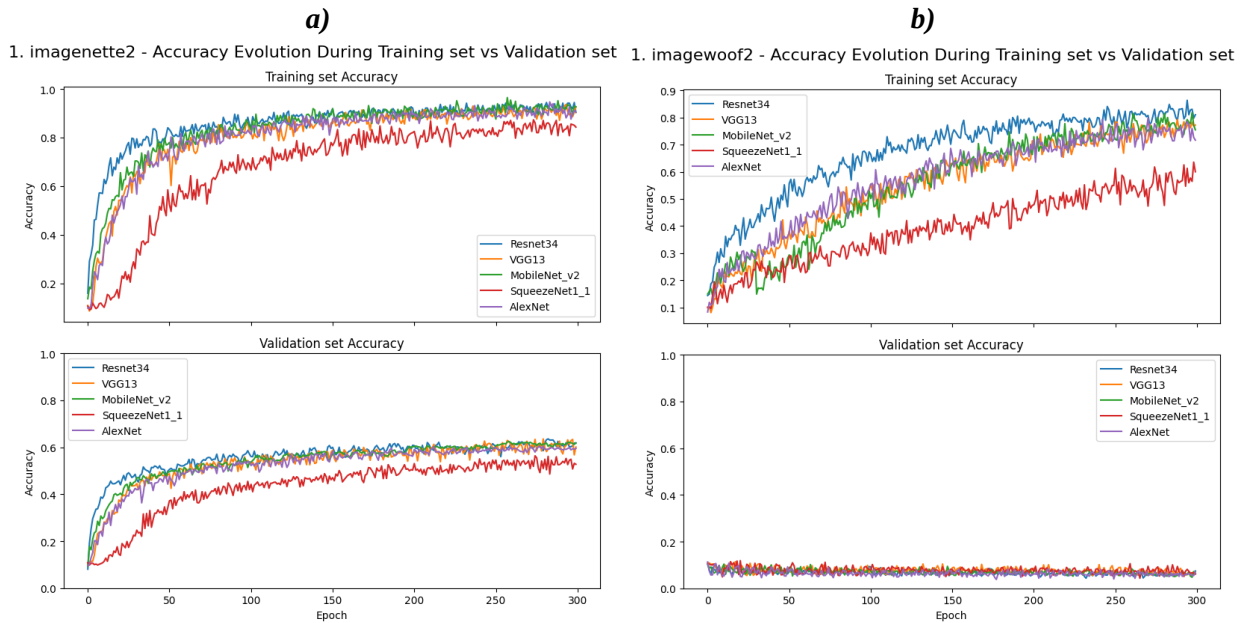


Figure 36. Classifiers trained with synthetic images accuracy progression.

a) Imagenette training and validation dataset, b) Imagewoof training and validation dataset

These plots provide us with a lot of information for both datasets. The most important thing to mention is that the Imagewoof classifiers trained with synthetic images are not able to generalize the information learned with the fake images and therefore they are not able to distinguish the real images. Although the fake images were of great quality this difference makes the classifiers not able to generalize. For imagenette we have an accuracy of around **64%** which tells us that the model can decently classify the real images but not as good as the classifiers trained with real images that had an accuracy of around **90%**. Now we will look at the **best results obtained for each model** and at what epoch they were reached. It is very important to understand this table that the accuracy of the results was done using the model version where the validation results were the highest at the shown epoch.

Table 4.4. Classifiers trained with synthetic images accuracy.

Databases	imagenette2					imagewoof2				
Metric	Top 1		Top 5		Epoch	Top 1		Top 5		Epoch
	train-1	val-1	train-5	val-5	Epoch	train-1	val-1	train-5	val-5	Epoch
AlexNet	0,909	0,598	0,99	0,897	280	0,111	0,057	0,471	0,471	1
MobileNet_v2	<u>0,932</u>	0,638	<u>0,995</u>	0,919	259	<u>0,235</u>	0,053	0,772	0,422	64
Resnet34	0,922	0,625	0,99	0,915	283	0,139	0,068	0,544	<u>0,529</u>	1
SqueezeNet1_1	0,863	0,554	0,992	0,879	275	0,187	0,064	0,724	0,412	21
VGG13	0,901	0,607	<u>0,995</u>	<u>0,92</u>	280	0,213	0,104	<u>0,787</u>	0,408	18

The best model for **Imagenette** is **MobileNet_v2** having an accuracy of **64%** hit at epoch 259, which tells us that the model was not able to generalize better from there and overfitting started. For Resnet and VGG13 which have accuracy of 62,5% and 60% respectively the best model was around epoch 280 which tells us that maybe they could hit a bit higher result if trained for more epochs.

For **Imagewoof**, as we show in the previous accuracy progression graph, the models are not able to classify the dog breeds correctly having the highest score with **VGG13** having an accuracy of **10,4%**. Another thing worth mentioning is the more epochs they are trained the less able they become to distinguish real images. As the best epochs for this models are pretty close to 0, the train results are also very bad as the model was not even able to distinguish the synthetic images when it hit the best validation accuracy, so we say that training the models with synthetic images made the models choose worse than if the dogs breed were chosen randomly, which is quite curious.

As for the models trained with real images, the accuracy per class will be shown. In Figure 4.17. for **imagenette**, and in Figure 4.18. for **Imagewoof**.

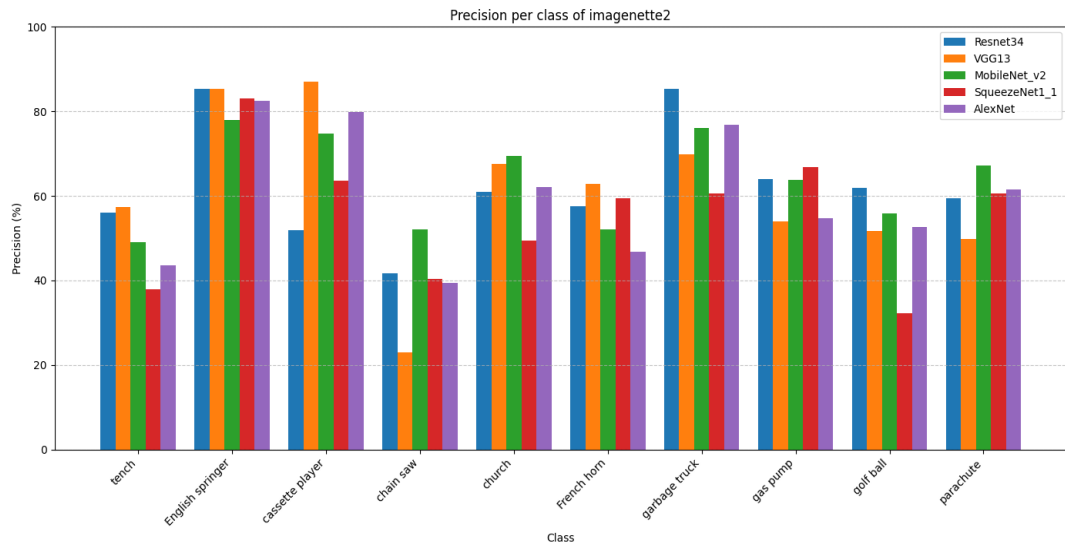


Figure 37. Accuracy per class of classifiers trained with synthetic images Imagenette.

As we can see for imagenette there are some classes that are clearly easier to classify than others having for **English springer** or **garbage truck** an accuracy of around **80%**. On the other hand classes like **chain saw** or **tench** are not classified well, having an accuracy of around **40%**. Although there are some differences, there is a balance in all the classes having at least 40%.

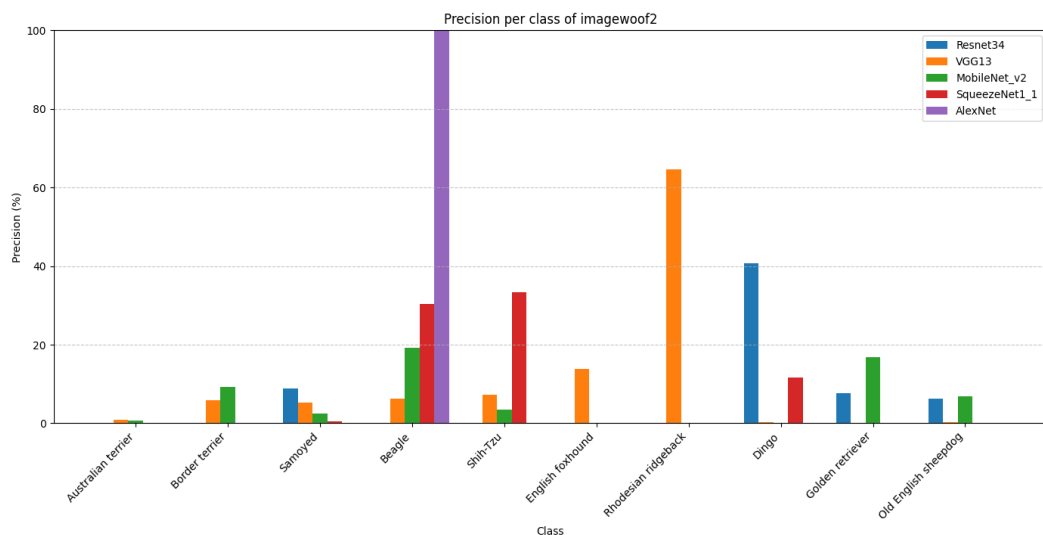


Figure 38. Accuracy per class of classifiers trained with synthetic images Imagewoof.

Here we can see the ability of classifying imagewoof classes is almost nonexistent.

4.3.4. Training time

Finally, we will compare the **training time** of each model in each dataset in Figure 4.19.

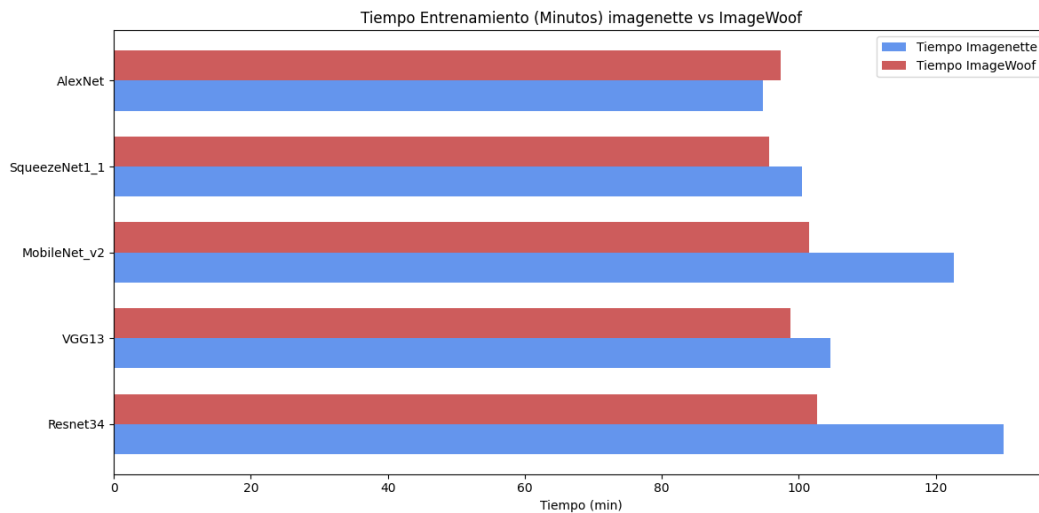


Figure 39. Training time for Imagewoof and Imagenette, classifiers trained with real values.

Now we can see all the models had almost the same training as the restrictions of the VRAM capacity were respected, we could have done the same with the classifiers trained with real images but it looked like and interesting thing to mention because in most of the computers the default setting is that is the VRAM is exceed it will use the RAM of the computer as a shared memory and it would load and unload data when needed. This may not cause a compilation error or running error, but it does cause longer training time required if not set carefully.

4.4. Comparison

In Table 4.5., the accuracy of the real classifiers and the synthetic classifiers will be shown together.

Table 4.5. Real and Synthetic classifiers comparison.

	Databases	imagenette2					imagewoof2				
		Top 1		Top 5		Epoch	Top 1		Top 5		Epoch
	Metric	train-1	val-1	train-5	val-5	Epoch	train-1	val-1	train-5	val-5	Epoch
REAL	AlexNet	0,977	0,887	0,999	0,985	293	0,918	0,778	0,993	0,973	282
	MobileNet_v2	0,99	0,919	<u>1</u>	0,992	292	0,98	0,838	<u>1</u>	0,979	275
	Resnet34	0,994	0,922	<u>1</u>	0,994	298	<u>0,984</u>	0,837	<u>1</u>	0,979	256
	SqueezeNet1_1	0,899	0,87	0,993	0,987	288	0,766	0,775	0,972	0,973	297
	VGG13	<u>0,986</u>	0,922	<u>1</u>	<u>0,993</u>	297	0,963	0,849	0,999	<u>0,985</u>	275
SYNTH	AlexNet	0,909	0,598	0,99	0,897	280	0,111	0,057	0,471	0,471	1
	MobileNet_v2	<u>0,932</u>	0,638	<u>0,995</u>	0,919	259	<u>0,235</u>	0,053	0,772	0,422	64
	Resnet34	0,922	0,625	0,99	0,915	283	0,139	0,068	0,544	<u>0,529</u>	1
	SqueezeNet1_1	0,863	0,554	0,992	0,879	275	0,187	0,064	0,724	0,412	21
	VGG13	0,901	0,607	<u>0,995</u>	<u>0,92</u>	280	0,213	0,104	<u>0,787</u>	0,408	18

Training models with synthetic images is not that easy as it will critically depend on the classification task difficulty. We can achieve decent results if the images are different from each other but when the classes are too similar, real images will have some details in the pictures that are useful for the classifier to determine. A true state is that if the synthetic images quality upgrade and the closer they look to real images, the easier it will be to make the classifiers trained with fake images learn.

4.5. Cost of training models

Deep Networks are expensive in terms of energy consumption and although for this project everything was run locally and the computer was not turned on the whole time, the increase in electricity consumption is worth mentioning. It was possible to get the energy consumption of these months in the house where the PC ran all the training and calculations. It is shown in Spanish as it is from a Spanish company named Iberdrola.

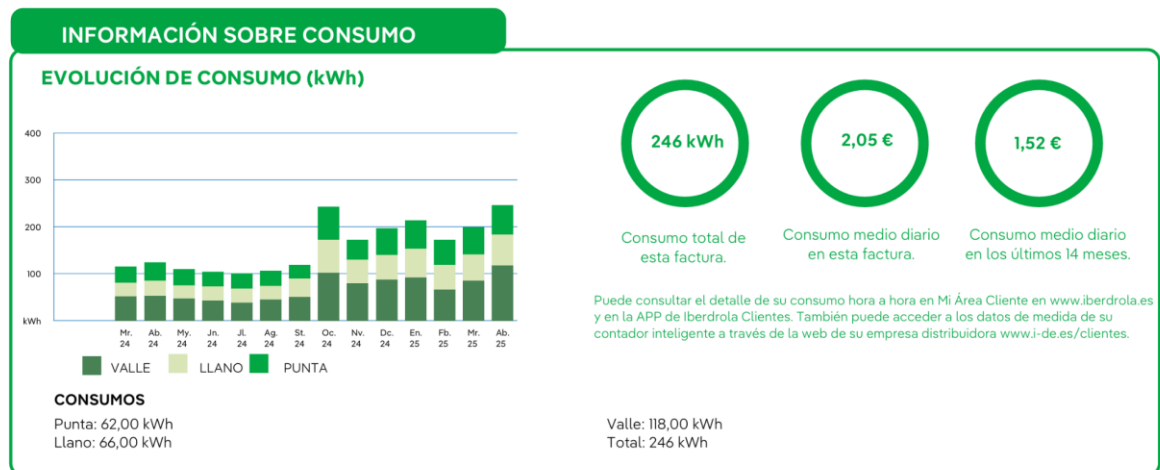


Figure 40. Iberdrola energy consumption.

Since October, when it first started, energy consumption has been duplicated and adding a total cost of around 30€ a month. For this project only one GPU was used but if multiple GPUs were used the energy cost would increase too so energy consumption is a direct cost of doing deep learning practices. For this project, the consumption of energy from the PC was equivalent to the consumption of 3 people living in a house.

Conclusions

This project was meant to evaluate the performance of classifiers, trained with synthetic images generated with GAN models, to assess how good they could become in recognizing real images compared to classifiers that were trained with real images. All of these experiments were done independently for two subsets of ImageNet that have 10 different classes each. The results varied depending on the dataset: Imagenette, which contains very different object categories, and Imagewoof, which includes various dog breeds with more subtle visual differences. The results varied depending on the dataset: **Imagenette**, which contains very different object categories, and **Imagewoof**, which includes various dog breeds with more subtle visual differences. The process of creating a GAN model from scratch was also presented, specifically using a DCGAN with a simple architecture to visualize how synthetic images are created.

Firstly, we trained the classifiers AlexNet, Resnet34, MobileNetv2, VGG13 and Squeezenet 1.1 with real images. The models achieved very good results classifying the images, with a maximum of **92%** hit by both **VGG13 and Resnet34 in Imagenette**. For **imagewoof** the best results obtained were **85%** hit by **VGG13**. These are solid results considering the difficulty of these databases that are known for having real-world variability in the pictures.

Afterwards, for the process of training a DCGAN model that generates images from scratch it became clear that multiple configurations were needed to optimize the results, as GAN training is highly sensitive to parameters like the learning rate. A key challenge was maintaining balance between the generator and discriminator, if one learns too quickly, training collapses. In addition, training one DCGAN model per class led to better outcomes as each model specialized in generating a specific category.

Although these DCGAN models created decent images they were far from looking realistic. Using other GANs was not in the objectives of the project because of the low-quality generated images other GANs were searched, installed and tested, to get better results. These GANs such as BigGAN, SAGAN, SNGAN were already trained on the full ImageNet database and for the evaluation of the generated images from these models the **FID** and **IS** metrics were applied. The model that got the **best results** was **BigGAN** achieving a FID of 96.868 in Imagenette and 83.534 in Imagewoof and therefore was the chosen one to train the new classifiers.

So finally, we trained the same five classifiers but now using the images BigGAN generated, and the best results obtained were a **64%** accuracy by **MobileNet_v2 on Imagenette** indicating that the results were inferior to real-data training but it still acquired some classification skills for the classes of the database. However, on **Imagewoof** the classifiers failed to generalize from synthetic to real images, hitting a best accuracy of **10% by VGG13**, which having 10 classes suggests predictions were nearly random.

What we learned in this project is that using real data to train models generally produces the best results. However, since real data can be difficult to obtain, using synthetic data to train models could be a viable alternative depending on the task. Synthetic images may be useful for training classifiers if the classification task is not too complex, the synthetic data is of high quality, achieving the highest possible accuracy is not critical, and real data is unavailable. On the other hand, generating high-quality synthetic images from scratch using GANs is a challenging task that requires models with very deep architectures and very careful training.

5.1. Future work

Train BigGAN on Smaller Datasets: BigGAN was originally trained on ImageNet, which contains 1000 classes, while in this project we only used its subsets, Imagenette and Imagewoof. Training a large GAN like BigGAN specifically on these smaller datasets could improve the quality of the generated synthetic images and potentially lead to better classifier performance.

Test with Higher-Quality Pretrained GANs: Load and evaluate a pretrained GAN that generates higher-quality images to observe how the quality of synthetic images influences the classifier's ability to distinguish between classes.

Extend to Other Modalities: Apply this approach of training classifiers with synthetic data and evaluating on real data to other domains such as text or audio, to assess whether similar patterns hold.

Use Simpler Datasets: Repeat this study using a simpler dataset with less real-world variability, which may help the synthetic images appear more realistic and aligned with the training objectives.

Hybrid Training with Partial Real Data: Since real training data can be expensive and hard to obtain, experiment with adding a small portion of real images (e.g., 10%) to the synthetic training set to enhance generalization.

Explore Alternative Generative Models: Conduct a similar study using other generative models like Autoencoders or Diffusion Models instead of GANs, to evaluate their potential in generating useful synthetic data for classifier training.

Bibliography

- [1] S. J. D. Prince, *Understanding Deep Learning*. Cambridge, MA: MIT Press, 2023.
- [2] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bull. Math. Biophys.*, vol. 5, no. 4, pp. 115–133, 1943. doi: 10.1007/BF02478259.
- [3] T. Salimans *et al.*, "Improved Techniques for Training GANs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, pp. 2234–2242, 2016.
- [4] PyTorch, "DCGAN Tutorial," *PyTorch Tutorials*, 2024. [Online]. Available: https://docs.pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html. [Accessed: Oct. 11, 2024].
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.
- [6] Y. Benny *et al.*, "Evaluation Metrics for Conditional Image Generation," *Int. J. Comput. Vis.*, vol. 129, pp. 1712–1731, 2021. doi: 10.1007/s11263-020-01424-w.
- [7] L. Alzubaidi *et al.*, "A survey on deep learning tools dealing with data scarcity: definitions, challenges, solutions, tips, and applications," *J. Big Data*, vol. 10, no. 46, Apr. 2023. [Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-023-00727-2>
- [8] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," *CoRR*, vol. abs/1511.06434, 2015.
- [9] Papers With Code, "Image Generation on ImageNet 256x256," <https://paperswithcode.com/sota/image-generation-on-imagenet-256x256> (accessed March 16, 2025).
- [10] A. Sauer, K. Schwarz, and A. Geiger, "StyleGAN-XL: Scaling StyleGAN to Large Diverse Datasets," GitHub, 2022. [Online]. Available: <https://github.com/autonomousvision/stylegan-xl>. [Accessed: March 19, 2025].

Appendices



Code

The full code is available in <https://github.com/CarlosVegadeAlba/DCGAN>, as the full code has around 5000 lines is not possible to show all in this appendix so it will be shown just some of the most important functions used for the development of this project that are:

1. Imports and global variables

```
import os
import re
import random
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.utils as vutils
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
import time
import copy
import gc
import pandas as pd
from torchvision.transforms import InterpolationMode, AutoAugmentPolicy
from collections import defaultdict
from IPython.display import display, Image as Img
from PIL import Image
import cv2
from torch.utils.data import TensorDataset, Dataset, ConcatDataset, DataLoader
from openpyxl import load_workbook
from openpyxl.styles import Font
from torchvision.models import resnet34, vgg13, mobilenet_v2, squeezenet1_1, alexnet
from torchmetrics.classification import Accuracy
from torchvision.datasets import ImageFolder
from torchvision.transforms.functional import to_pil_image
from pytorch_pretrained_gans import make_gan
from pytorch_pretrained_biggan.utils import one_hot_from_names, truncated_noise_sample
from torchmetrics.image.fid import FrechetInceptionDistance
from torchmetrics.image.inception import InceptionScore
import torch.nn.functional as Functional
from torch.optim.lr_scheduler import CosineAnnealingLR

# Set random seed for reproducibility
manualSeed = 999
# manualSeed = random.randint(1, 10000) # use if you want new results
print("Random Seed: ", manualSeed)
random.seed(manualSeed)
torch.manual_seed(manualSeed)
torch.use_deterministic_algorithms(True) # Needed for reproducible results in DCGAN
print(torch.cuda.is_available()) # True
print(torch.version.cuda) # '11.8'
print(torch.cuda.get_device_name(0)) # 'NVIDIA GeForce RTX 4070 SUPER'
print(torch.backends.cudnn.version()) # 8700
print(torch.backends.cudnn.enabled) # True

# Number of GPUs available. Use 0 for CPU mode.
NGPU = 1
device = torch.device("cuda:0" if (torch.cuda.is_available() and NGPU > 0) else "cpu")

# Set random seed for reproducibility
manualSeed = 999
```

```

#manualSeed = random.randint(1, 10000) # use if you want new results
print("Random Seed: ", manualSeed)
random.seed(manualSeed)
torch.manual_seed(manualSeed)
torch.use_deterministic_algorithms(True) # Needed for reproducible results

print(torch.cuda.is_available())    # True
print(torch.version.cuda)           # '11.8'
print(torch.cuda.get_device_name(0)) # 'NVIDIA GeForce RTX 4070 SUPER'
print(torch.backends.cudnn.version()) # 8700
print(torch.backends.cudnn.enabled)  # True

# Number of GPUs available. Use 0 for CPU mode.
NGPU = 1
device = torch.device("cuda:0" if (torch.cuda.is_available() and NGPU > 0) else "cpu")

IMAGENETTE = "imagenette2"
IMAGEWOOF = "imagewoof2"
DATABASES = [IMAGENETTE, IMAGEWOOF]
GENERATED = os.path.join("data", "generated")
datarootImagenette = os.path.join("data", "databases", IMAGENETTE)
datarootImagewoof = os.path.join("data", "databases", IMAGEWOOF)
path_datarootImagewoofTrain = os.path.join(datarootImagewoof, "train")
path_datarootImagenetteTrain = os.path.join(datarootImagenette, "train")
path_datarootImagewoofVal = os.path.join(datarootImagewoof, "val")
path_datarootImagenetteVal = os.path.join(datarootImagenette, "val")
paths_dataroot = {IMAGENETTE: {'train': path_datarootImagenetteTrain, 'val': path_datarootImagenetteVal}, IMAGEWOOF: {'train': path_datarootImagewoofTrain, 'val': path_datarootImagewoofVal}}
PLOTS = os.path.join(GENERATED, "plots")
MODELS = os.path.join(GENERATED, "models")
MODELS_CLASSIFIERS = os.path.join(MODELS, "classifiers")
TRANSFORMED_DATASET = os.path.join(GENERATED, "transformed_datasets")
NUM_IMAGES_FOR_STEP = 32
ROUNDS_PER_SAVE_IMAGES = 1
ROUNDS_PER_SAVE_MODELS = 300
NUM_EPOCHS = 300
MEAN=[0.485, 0.456, 0.406]
STD=[0.229, 0.224, 0.225]
# Number of channels in the training images. For color images this is 3
NC = 3
# Beta1 hyperparameter for Adam optimizers
beta1 = 0.5
# Learning rates values
LEARNING_RATES = [ 0.0002, 0.0004, 0.0006]
NZ_VALUES = [100]
IMG_SIZES = [64]
# Batch size during training for image size
BATCH_SIZE = {64: 256}
TRUNCATION = 0.7

# IMAGENETTE
classifierImagenette = {
    "tench": "n01440764", # Type of fish
    "English springer": "n02102040", # Type of dog
    "cassette player": "n02979186",
    "chain saw": "n03000684",
    "church": "n03028079",
    "French horn": "n03394916",
    "garbage truck": "n03417042",
    "gas pump": "n03425413",
    "golf ball": "n03445777",
    "parachute": "n03888257"
}

classifierImagenette_inv = {valor: clave for clave, valor in classifierImagenette.items()}

id_to_index_imagenette = {
    imagenet_id: idx for idx, imagenet_id in enumerate(sorted(classifierImagenette.values()))
}

# IMAGEWOOF
classifierImagewoof = {
    "Australian terrier": "n02086240",
    "Border terrier": "n02087394",
    "Samoyed": "n02088364",

```



```

        "Beagle": "n02089973",
        "Shih-Tzu": "n02093754",
        "English foxhound": "n02096294",
        "Rhodesian ridgeback": "n02099601",
        "Dingo": "n02105641",
        "Golden retriever": "n02111889",
        "Old English sheepdog": "n02115641"
    }

    classifierImagewoof_inv = {valor: clave for clave, valor in classifierImagewoof.items()}

    id_to_index_imagewoof = {
        imagenet_id: idx for idx, imagenet_id in enumerate(sorted(classifierImagewoof.values()))
    }

    # generalization
    classifierFromName = {
        IMAGENETTE: classifierImagenette,
        IMAGEWOOF: classifierImagewoof
    }

    # generalization
    classifierFromNameInv = {
        IMAGENETTE: classifierImagenette_inv,
        IMAGEWOOF: classifierImagewoof_inv
    }

    # Diccionario de nombres de clases
    CLASS_NAMES = {
        IMAGENETTE: [
            'tench', 'English springer', 'cassette player', 'chain saw',
            'church', 'French horn', 'garbage truck', 'gas pump',
            'golf ball', 'parachute'
        ],
        IMAGEWOOF: [
            'Australian terrier', 'Border terrier', 'Samoyed', 'Beagle',
            'Shih-Tzu', 'English foxhound', 'Rhodesian ridgeback',
            'Dingo', 'Golden retriever', 'Old English sheepdog'
        ]
    }

    # Diccionario con índices numéricos de las clases en ImageNet
    IMAGENET_INDICES = {
        IMAGENETTE: {
            'tench': 0,
            'English springer': 217,
            'cassette player': 482,
            'chain saw': 491,
            'church': 497,
            'French horn': 566,
            'garbage truck': 569,
            'gas pump': 571,
            'golf ball': 574,
            'parachute': 701
        },
        IMAGEWOOF: {
            'Australian terrier': 193,
            'Border terrier': 182,
            'Samoyed': 258,
            'Beagle': 162,
            'Shih-Tzu': 155,
            'English foxhound': 167,
            'Rhodesian ridgeback': 159,
            'Dingo': 273,
            'Golden retriever': 207,
            'Old English sheepdog': 229
        }
    }

    # Diccionario con vectores one-hot (para BigGAN)
    CLASS_VEC = {
        dataset: one_hot_from_names(CLASS_NAMES[dataset], batch_size=len(CLASS_NAMES[dataset]))
        for dataset in CLASS_NAMES
    }

```

```

# Diccionario con tensores de índices numéricos (para StudioGAN)
CLASS_INDICES = {
    dataset: torch.tensor(
        [IMAGENET_INDICES[dataset][name] for name in CLASS_NAMES[dataset]],
        dtype=torch.long
    )
    for dataset in CLASS_NAMES

}

TRANSFORM_VAL = transforms.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

TRANSFORM_TRAIN = transforms.Compose([
    # Recorte aleatorio escalado (manteniendo aspecto) a 224x224
    transforms.RandomResizedCrop(
        size=224, scale=(0.4, 1.0), ratio=(1.0, 1.0),
        interpolation=InterpolationMode.BICUBIC
    ),
    # Flip horizontal aleatorio
    transforms.RandomHorizontalFlip(p=0.5),
    # AutoAugment con la política de ImageNet
    transforms.AutoAugment(
        policy=AutoAugmentPolicy.IMAGENET,
        interpolation=InterpolationMode.BILINEAR
    ),
    # Variaciones aleatorias de color
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
        saturation=0.2, hue=0.02),
    # Blur
    transforms.GaussianBlur(kernel_size=(3, 3), sigma=(0.01, 0.4)),
    # Sharpness
    transforms.RandomAdjustSharpness(sharpness_factor=1.5, p=0.5), # Corregido aquí
    # Conversión a tensor [0,1]
    transforms.ToTensor(),
    # (Opcional) Borrado aleatorio de un parche
    # transforms.RandomErasing(p=0.1),
    # Normalización con media y desvío estándar de ImageNet
    transforms.Normalize(MEAN, STD)
])

```

2. Create Preloaded dataset

```

class PreloadedImageFolder(Dataset):
    def __init__(self, root, label, transform=None):
        super().__init__()
        self.root = root
        self.transform = transform
        self.preloaded_data = []
        self.label = torch.tensor(label) # Guardar la etiqueta de la clase

    # Obtener solo archivos de imagen en la carpeta
    self.samples = [os.path.join(root, f) for f in os.listdir(root) if f.lower().endswith(('jpg', 'jpeg', 'png'))]

    for path in self.samples:
        sample = Image.open(path).convert("RGB") # Cargar imagen

        if self.transform:
            sample = self.transform(sample) # Aplicar la transformación

        self.preloaded_data.append((sample, self.label)) # Guardar imagen

    def __len__(self):
        return len(self.preloaded_data)

    def __getitem__(self, index):
        return self.preloaded_data[index]

```

2. Create Dynamic dataset with DataAugmentations

```
class PreloadedImageFolderTransformations(Dataset):
    def __init__(self, root, label, transform):
        super().__init__()
        self.root = root
        self.transform = transform
        self.label = torch.tensor(label) # Guardar la etiqueta de la clase
        # Obtener solo archivos de imagen en la carpeta
        self.samples = [os.path.join(root, f) for f in os.listdir(root) if f.lower().endswith(('jpg', 'jpeg', 'png'))]

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, index):
        path = self.samples[index]
        sample = Image.open(path).convert("RGB") # Cargar imagen
        sample = self.transform(sample) # Aplicar la transformación
        return (sample, self.label)
```

3. Models DCGAN - Generator64 and Discriminator64

```
class PreloadedImageFolderTransformations(Dataset):
    def __init__(self, root, label, transform):
        super().__init__()
        self.root = root
        self.transform = transform
        self.label = torch.tensor(label) # Guardar la etiqueta de la clase
        # Obtener solo archivos de imagen en la carpeta
        self.samples = [os.path.join(root, f) for f in os.listdir(root) if f.lower().endswith(('jpg', 'jpeg', 'png'))]

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, index):
        path = self.samples[index]
        sample = Image.open(path).convert("RGB") # Cargar imagen
        sample = self.transform(sample) # Aplicar la transformación
        return (sample, self.label)
```

4. Models DCGAN - Generator64 and Discriminator64

```
# Discriminator Code
class Discriminator_64(nn.Module):
    def __init__(self, ngpu, ndf):
        super(Discriminator_64, self).__init__()
        self.ngpu = ngpu
        self.ndf = ndf
        self.main = nn.Sequential(
            # input is `(NC) x 64 x 64`
            nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, bias=True)
            nn.Conv2d(NC, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. `(ndf) x 32 x 32`
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. `(ndf*2) x 16 x 16`
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. `(ndf*4) x 8 x 8`
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. `(ndf*8) x 4 x 4`
            nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
```

```

        nn.Sigmoid(),
    )

    def forward(self, input):
        return self.main(input)

# Generator Code
class Generator_64(nn.Module):
    def __init__(self, ngpu, nz, ngf):
        super(Generator_64, self).__init__()
        self.ngpu = ngpu
        self.nz = nz
        self.ngf = ngf
        self.main = nn.Sequential(
            # input is Z, going into a convolution
            nn.ConvTranspose2d(nz, ngf * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            # state size. ``(ngf*8) x 4 x 4``
            nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            # state size. ``(ngf*4) x 8 x 8``
            nn.ConvTranspose2d(ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),
            # state size. ``(ngf*2) x 16 x 16``
            nn.ConvTranspose2d(ngf * 2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            # state size. ``(ngf) x 32 x 32``
            nn.ConvTranspose2d(ngf, NC, 4, 2, 1, bias=False),
            nn.Tanh(),
            # state size. ``(NC) x 64 x 64``
        )

    def forward(self, input):
        return self.main(input)

# custom weights initialization called on ``netG`` and ``netD``
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

```

5. Train DCGAN models

```

def save_model(filename, model, optimizer, scheduler, epoch, losses, accuracy, img_list=None, num_images=None, trained_time=None):
    if scheduler is not None:
        scheduler = scheduler.state_dict()

    torch.save({
        'model_state_dict': model.state_dict(),
        'optimizer': optimizer,
        'scheduler_state_dict': scheduler,
        'epoch': epoch,
        'losses': losses,
        'accuracy': accuracy,
        'img_list': img_list,
        'num_images': num_images,
        'trainedTime': trained_time,
    }, filename)

```

```
def train(num, modelD_filename, netD, optimizerD, schedulerD, modelG_filename, netG, optimizerG, schedulerG, dataloader, trained_epochs,
num_epochs, image_size, nz, fixed_noise, discriminatorUpdate, generatorUpdate, D_accuracy=None, G_accuracy=None, D_losses=None,
G_losses=None, img_list=None, num_images=None, trained_time=None):
```

```
    # Lists to keep track of progress
```

```
    if img_list is None:
```

```
        # Crear un tensor vacío con forma (0, 3, 64, 64)
```

```
        img_list = torch.empty((0, 3, image_size, image_size))
```

```
    if num_images is None:
```

```
        num_images = 0
```

```
def expand_list(lst, num_epochs, update=1):
```

```
    if lst is None:
```

```
        return [None] * (num_epochs // update)
```

```
    else:
```

```
        if len(lst) < (num_epochs // update):
```

```
            lst.extend([None] * ((num_epochs // update) - len(lst)))
```

```
        return lst
```

```
trained_time = expand_list(trained_time, num_epochs, generatorUpdate)
```

```
D_accuracy = expand_list(D_accuracy, num_epochs, discriminatorUpdate)
```

```
G_accuracy = expand_list(G_accuracy, num_epochs, generatorUpdate)
```

```
D_losses = expand_list(D_losses, num_epochs, discriminatorUpdate)
```

```
G_losses = expand_list(G_losses, num_epochs, generatorUpdate)
```

```
last_errD = 0.0
```

```
last_D_x = 0.0
```

```
last_D_G_z1 = 0.0
```

```
netG = netG.to(device)
```

```
netD = netD.to(device)
```

```
# For each epoch
```

```
for i, epoch in enumerate(range(trained_epochs, num_epochs)):
```

```
    # For each batch in the dataloader
```

```
    start_time = time.time()
```

```
    for data in dataloader:
```

```
        #####
```

```
        # (1) Update D network: maximize  $\log(D(x)) + \log(1 - D(G(z)))$ 
```

```
        #####
```

```
        if ((epoch % discriminatorUpdate) == 0 or (epoch == trained_epochs)):
```

```
            ## Train with all-real batch
```

```
            netD.zero_grad()
```

```
            # Format batch
```

```
            real_cpu = data[0].to(device)
```

```
            b_size = real_cpu.size(0)
```

```
            label = torch.full((b_size,), real_label, dtype=torch.float, device=device)
```

```
            # Forward pass real batch through D
```

```
            output = netD(real_cpu).view(-1)
```

```
            # Calculate loss on all-real batch
```

```
            errD_real = criterion(output, label)
```

```
            # Calculate gradients for D in backward pass
```

```
            errD_real.backward()
```

```
            D_x = output.mean().item()
```

```
            ## Train with all-fake batch
```

```
            # Generate batch of latent vectors
```

```
            noise = torch.randn(b_size, nz, 1, 1, device=device)
```

```
            # Generate fake image batch with G
```

```
            fake = netG(noise)
```

```
            label.fill_(fake_label)
```

```
            # Classify all fake batch with D
```

```
            output = netD(fake.detach()).view(-1)
```

```
            # Calculate D's loss on the all-fake batch
```

```
            errD_fake = criterion(output, label)
```

```
            # Calculate the gradients for this batch, accumulated (summed) with previous gradients
```

```
            errD_fake.backward()
```

```
            D_G_z1 = output.mean().item()
```

```
            # Compute error of D as sum over the fake and the real batches
```

```
            errD = errD_real + errD_fake
```

```
            # Update D
```

```
            optimizerD.step()
```

```
            # Guardar los valores actuales
```

```
            last_errD = errD.item()
```

```
            last_D_x = D_x
```

```
            last_D_G_z1 = D_G_z1
```



```

    if schedulerD is not None:
        schedulerD.step() # Update the learning rate based on the scheduler
else:
    noise = torch.randn(b_size, nz, 1, 1, device=device)
    # Generate fake image batch with G
    fake = netG(noise)
    label.fill_(fake_label)
    # Recuperar valores anteriores si no se actualiza D
    errD = torch.tensor(last_errD)
    D_x = last_D_x
    D_G_z1 = last_D_G_z1

#####
# (2) Update G network: maximize log(D(G(z)))
#####
if ((epoch % generatorUpdate) == 0 or (epoch == trained_epochs)):
    netG.zero_grad()
    label.fill_(real_label) # fake labels are real for generator cost
    # Since we just updated D, perform another forward pass of all-fake batch through D
    output = netD(fake).view(-1)
    # Calculate G's loss based on this output
    errG = criterion(output, label)
    # Calculate gradients for G
    errG.backward()
    D_G_z2 = output.mean().item()
    # Update G
    optimizerG.step()
    if schedulerG is not None:
        schedulerG.step() # Update the learning rate based on the scheduler
else:
    # No se actualiza G: se usan los valores anteriores
    errG = torch.tensor(G_losses[-1]) if G_losses else torch.tensor(0.0)
    D_G_z2 = D_G_z2 if 'D_G_z2' in locals() else 0.0

# Save Losses for plotting later
G_losses[i//generatorUpdate] = errG.item()
G_accuracy[i//generatorUpdate] = D_G_z1

D_losses[i//discriminatorUpdate] = errD.item()
D_accuracy[i//discriminatorUpdate] = D_x
# END OF EPOCH
if (torch.cuda.is_available() and ((torch.cuda.memory_allocated() / 1024**2 > 10000) or (torch.cuda.memory_reserved() / 1024**2 >
10000))) : # Check if the gpu is too full, reduce batch size
    print("***** WARNING *****")
    print(f"CUDA memory allocated: {torch.cuda.memory_allocated() / 1024**2:.2f} MB")
    print(f"CUDA memory reserved: {torch.cuda.memory_reserved() / 1024**2:.2f} MB")
# Output training stats
if schedulerG is not None:
    schedulerG.step()

# Save the model state
if (((epoch+1) % ROUNDS_PER_SAVE_IMAGES) == 0 or ((epoch+1) == num_epochs)):
    print(f"[{epoch+1}]\tLoss_D: %.4f\tLoss_G: %.4f\tD(x): %.4f\tD(G(z)): %.4f / %.4f" % (epoch+1, num_epochs, errD.item(), errG.item(), D_x,
D_G_z1, D_G_z2))
    with torch.no_grad():
        fake = netG(fixed_noise).detach().cpu()
        # Normalizar imágenes de [-1,1] a [0,1]
        fake = (fake + 1) / 2
        img_list = torch.cat((fake, img_list), dim=0) # Concat the new images
        num_images += 1

if (((epoch+1) % ROUNDS_PER_SAVE_MODELS) == 0 or ((epoch+1) == num_epochs)):
    # Save the model state
    save_model(modelD_filename, netD, optimizerD, schedulerD, epoch+1, D_losses, D_accuracy)
    save_model(modelG_filename, netG, optimizerG, schedulerG, epoch+1, G_losses, G_accuracy, img_list, num_images, trained_time)

trained_time[epoch - 1] = (time.time() - start_time)

# END OF TRAINING
print(f"\tModel {num} trained for {num_epochs} epochs. Time spent: {sum(t for t in trained_time if t is not None):.2f} seconds")

```

6. Show DCGAN image generation evolution

```

INCREMENT = 75
def show_image_results(models, img_size, increment=None):
    if SHOW_PROGRESS_IMAGES:
        for num, model in enumerate(models.keys(), 1):
            # Crear un tensor vacío con forma (0, 3, 64, 64)
            total_models = len(models[model])

            print("\n\nShowing Results of Model", num)
            for i in range(total_models):
                _, modelG_filename, datasetTitle = models[model][i]

                if os.path.isfile(modelG_filename):
                    checkpoint = torch.load(modelG_filename)
                    img_list = torch.empty((0, 3, img_size, img_size))
                    model_images = checkpoint['img_list']
                    #num_images = checkpoint['num_images']
                    epochs = checkpoint['epoch']
                    # Only add images if num_images % increment == 0

                def get_values_to_plot(N, M, X):
                    """
                    N: número total de iteraciones (ej. 500)
                    M: cada cuántas iteraciones se guardó un valor (ej. 50)
                    X: cada cuántas iteraciones quiero mostrar un resultado (ej. 100)

                    Devuelve una lista de índices que se pueden usar para acceder
                    a los elementos correctos en un array que guarda datos cada M iteraciones,
                    para así mostrar el valor correspondiente a cada X iteraciones reales.
                    """
                    return [i // M for i in range(0, N + 1, X) if i % M == 0]

            indices_to_plot = get_values_to_plot(epochs, ROUNDS_PER_SAVE_IMAGES, increment)

            for i in indices_to_plot:
                # img_list = torch.cat((img_list, model_images[i].unsqueeze(0)), dim=0)
                fromIndex = i*NUM_IMAGES_FOR_STEP
                toIndex = (i+1)*NUM_IMAGES_FOR_STEP
                if fromIndex > NUM_IMAGES_FOR_STEP:
                    fromIndex -= NUM_IMAGES_FOR_STEP
                toIndex -= NUM_IMAGES_FOR_STEP
                img_list = torch.cat((img_list, model_images[fromIndex: toIndex]), dim=0)
            # img_list = torch.cat((img_list, model_images), dim=0)
            grid_img = vutils.make_grid(img_list, nrow=NUM_IMAGES_FOR_STEP, normalize=True)
            np_img = grid_img.permute(1, 2, 0).numpy()

            # Usa fig + ax para mantener control total sobre la figura
            fig, ax = plt.subplots(figsize=(20, 20))
            ax.imshow(np_img)
            ax.axis('off')
            if IMAGENETTE in datasetTitle:
                classifier = classifierImagenette_inv
                datasetName = IMAGENETTE
            elif IMAGEWOOF in datasetTitle:
                classifier = classifierImagewoof_inv
                datasetName = IMAGEWOOF
            else:
                print("Invalid dataset")
                break

            if os.path.isfile(modelG_filename):
                checkpoint = torch.load(modelG_filename)

            pattern = r"nz=(\d+)_lrD=(\d+)_lrG=(\d+)_imgSize=(\d+)_uD=(\d+)_uG=(\d+)_modData=(\d+)_ModelG-([a-z0-9]+)"

            match = re.match(pattern, os.path.basename(modelG_filename))

            if match:
                nz = int(match.group(1))
                lrD = float(match.group(2))
                lrG = float(match.group(3))
                imgSize = int(match.group(4))
                uD = int(match.group(5))

```

```

        uG = int(match.group(6))
        modData = int(match.group(7))
        n = match.group(8)
    else:
        print(os.path.basename(modelG_filename))
        print("La cadena no coincide con el patrón.")
    else:
        print("Model not found")

# Añadir título a la figura
ax.set_title(rf"{num}.{datasetName} Generator evolution during training for ${bf{{{classifier[n]}}}}$. nz={nz}, lrD={lrD}, lrG={lrG},
uG={uG}, uD={uD}, imgSize={imgSize}, modData={modData}", fontsize=16)

# Añadir texto por fila (una etiqueta por iteración)
numbers = list(indices_to_plot)[:1]
num_rows = len(list(indices_to_plot))
img_height = img_size # o 128 si estás generando imágenes más grandes

for i in range(num_rows):
    y = i * img_height + img_height // 2
    ax.text(-10, y, f"Epoch {numbers[i]*ROUNDS_PER_SAVE_IMAGES}", va='center', ha='right',
            fontsize=12, color='white', backgroundcolor='black')

plt.tight_layout()
plt.show()

```

7. Loading GAN pretrained models

```

G_BigGan = make_gan(gan_type='biggan') # BigGAN preentrenado (256px)
G_BigGanDeep = make_gan(gan_type='biggan', model_name='biggan-deep-512')
G_Sagan = make_gan(gan_type='studiogan', model_name='SAGAN') # asegúrate que el modelo es condicional
G_Sngan = make_gan(gan_type='studiogan', model_name='SNGAN') # asegúrate que el modelo es condicional

```

```

pretrained_models = [
    (G_BigGan, 'BigGan'),
    (G_BigGanDeep, 'biggan-deep-512'),
    (G_Sagan, 'SAGAN'),
    (G_Sngan, 'SNGAN')
]

```

```

pretrained_modelsFromName = {
    'BigGan': G_BigGan,
    'biggan-deep-512': G_BigGanDeep,
    'SAGAN': G_Sagan,
    'SNGAN': G_Sngan,
}

```

8. Evaluate FID y IS of every GAN loaded

```

def evaluate_gans_imgGeneration(pretrained_models, databases, paths_dataroot, truncation=0.4, num_batches=10, batch_size=32, device='cuda'):
    if EVALUATE_IMAGES_GANS:
        filename = os.path.join(PLOTS, 'GANsEvaluation.xlsx')

        if os.path.exists(filename):
            df = pd.read_excel(filename, header=[0, 1], index_col=0)

            # Eliminar columnas que se llamen 'Metric' si existen
            df = df.loc[:, df.columns.get_level_values(0) != 'Metric']

            print("Loaded evaluation from Excel:")
            return df

        allModelsEvaluation = {}
        for database in databases:
            allModelsEvaluation[database] = {}

            # Dataset y loader para imágenes reales
            real_dataset = ImageFolder(paths_dataroot[database]['val'], transform=TRANSFORM_VAL)
            loader_real = DataLoader(real_dataset, batch_size=batch_size, shuffle=False)

```

```

for modelo_gan, modelName in pretrained_models:
    # Inicializar métricas
    fid_metric = FrechetInceptionDistance(feature=2048).to(device)
    is_metric = InceptionScore().to(device)

    # Procesar imágenes reales
    with torch.no_grad():
        for imgs, _ in loader_real:
            # Deshacer normalización
            imgs_uint8 = copy.deepcopy(imgs)
            for c in range(3):
                imgs_uint8[:, c] = imgs_uint8[:, c] * STD[c] + MEAN[c]
            imgs_uint8 = torch.clamp(imgs_uint8 * 255, 0, 255).type(torch.uint8)
            fid_metric.update(imgs_uint8.to(device), real=True)

    # Función interna para generar imágenes sintéticas usando el GAN
    def generar_imagenes_gan():
        modelo_gan.to(device)

        classes = get_classes_from_model_name(modelName, database).to(device)
        num_classes = len(classes)

        # Repetir clases para ajustar al batch_size
        classes_expanded = classes.repeat_interleave(batch_size // num_classes + 1, dim=0)[:batch_size]

        noise_vec = truncated_noise_sample(truncation=truncation, batch_size=batch_size)
        noise_vec = torch.from_numpy(noise_vec).to(device)

        with torch.no_grad():
            output = modelo_gan(noise_vec, classes_expanded)

        output = (output + 1) / 2 # [-1,1] -> [0,1]
        output = torch.clamp(output, 0, 1)

        # Redimensionar correctamente con interpolación
        output_resized = Functional.interpolate(output, size=(224, 244), mode='bilinear', align_corners=False)
        output_resized = (output_resized * 255).type(torch.uint8)

        del classes, noise_vec
        torch.cuda.empty_cache()
        torch.cuda.ipc_collect()
        gc.collect()

    return output_resized

    # Generar imágenes y actualizar métricas
    with torch.no_grad():
        for _ in range(num_batches):
            imgs_gen = generar_imagenes_gan()
            fid_metric.update(imgs_gen.to(device), real=False)
            is_metric.update(imgs_gen.to(device))

    # Calcular métricas finales
    fid_value = fid_metric.compute().item()
    is_mean, is_std = is_metric.compute()

    # Guardar resultados
    allModelsEvaluation[database][modelName] = [fid_value, float(is_mean), float(is_std)]

    # Opcionalmente, imprimir resultados
    print(f"Modelo: {modelName} | Base de datos: {database}")
    print(f"FID: {fid_value:.4f}")
    print(f>Inception Score: {float(is_mean):.4f} ± {float(is_std):.4f}\n")
    del modelo_gan, fid_metric, is_metric
    torch.cuda.empty_cache()
    gc.collect()

dict_to_dataframe(allModelsEvaluation, ['fid', 'is_mean', 'is_std'], os.path.join(filename))

```

```
evaluate_gans_imgGeneration(pretrained_models, DATABASES, paths_dataroot)
```

9. Create Dynamic dataset with GAN synthetic images

```
class ImagenesGeneradasDatasetDinamic(Dataset):
    def __init__(self, netG, modelName, database, transform, num_images=IMAGES_PER_CLASS, truncation=TRUNCATION):
        self.netG = netG
        self.truncation = truncation
        self.modelName = modelName.lower()
        self.database = database
        self.num_images = num_images
        self.transform = transform

    def __len__(self):
        return self.num_images

    def __getitem__(self, idx):
        device = 'cuda' if torch.cuda.is_available() else 'cpu'

        if self.modelName in ['sagan studiogan', 'sagan', 'sngan studiogan', 'sngan']:
            chosen_class = torch.randint(len(CLASS_INDICES[self.database]), (1,), dtype=torch.long)
            class_label = CLASS_INDICES[self.database][chosen_class].to(device)
            label = chosen_class.item()

        elif self.modelName in ['biggan', 'biggan-deep-512']:
            chosen_index = np.random.choice(len(CLASS_VEC[self.database]), 1)
            class_label = torch.from_numpy(CLASS_VEC[self.database][chosen_index]).to(device)
            label = chosen_index.item()

        noise_vec = truncated_noise_sample(truncation=self.truncation, batch_size=1)
        noise_vec = torch.from_numpy(noise_vec).to(device)

        self.netG.to(device)
        with torch.no_grad():
            output = self.netG(noise_vec, class_label)

        image = output.cpu().squeeze(0)
        image = (image + 1) / 2 # [-1,1] -> [0,1]
        image = transforms.ToPILImage()(image)
        if self.transform:
            image = self.transform(image)

        return image, label

def createFakeDataloader(model, modelName, truncation):
    dataloader_classifiersTrainFake = {}
    for database in DATABASES:
        datasetsFakeImages = ImagenesGeneradasDatasetDinamic(model, modelName, database, TRANSFORM_TRAIN, truncation=truncation)
        # All dataloaders ready
        dataloader_classifiersTrainFake[database] = DataLoader(datasetsFakeImages, batch_size=BATCH_SIZE_CLASSIFIER, shuffle=True)
    return dataloader_classifiersTrainFake
```

10. Classifiers models

```
classifiersModels = {}
```

```
# Step 1: Initialize models
# model, lr
classifiersModels['Resnet34'] = [resnet34(weights=None, num_classes=10), 1e-4]
classifiersModels['VGG13'] = [vgg13(weights=None, num_classes=10), 1e-4]
classifiersModels['MobileNet_v2'] = [mobilenet_v2(weights=None, num_classes=10), 5e-4]
classifiersModels['SqueezeNet1_1'] = [squeezenet1_1(weights=None, num_classes=10), 3e-4]
classifiersModels['AlexNet'] = [alexnet(weights=None, num_classes=10), 1e-4]
```


11. Train classifiers models

```
def save_classifier_model(model, optimizer, scheduler, epoch, best_epoch, losses, filename, trained_time, accuracy_list, accuracy_listVal):
    if scheduler is not None:
        scheduler = scheduler.state_dict()

    torch.save({
        'model_state_dict': model,
        'optimizer': optimizer,
        'scheduler_state_dict': scheduler,
        'epoch': epoch,
        'best_epoch': best_epoch,
        'losses': losses,
        'accuracy_list': accuracy_list,
        'accuracy_listVal': accuracy_listVal,
        'trainedTime': trained_time,
    }, filename)

def train_classifier_model(model_filename, model, optimizer, scheduler, dataloader, trained_epochs, num_epochs, device, best_epoch=None,
dataloaderVal=None, num_classes=10, losses=None, accuracy_list=None, accuracy_listVal=None, trained_time=None):
    criterion = nn.CrossEntropyLoss()
    if losses is None:
        losses = [None] * num_epochs
    else:
        if len(losses) < num_epochs:
            losses.extend([None] * (num_epochs - len(losses)))

    if trained_time is None:
        trained_time = [None] * num_epochs
    else:
        if len(trained_time) < num_epochs:
            trained_time.extend([None] * (num_epochs - len(trained_time)))

    if accuracy_list is None:
        accuracy_list = [None] * num_epochs
    else:
        if len(accuracy_list) < num_epochs:
            accuracy_list.extend([None] * (num_epochs - len(accuracy_list)))

    if accuracy_listVal is None:
        accuracy_listVal = [None] * num_epochs
    else:
        if len(accuracy_listVal) < num_epochs:
            accuracy_listVal.extend([None] * (num_epochs - len(accuracy_listVal)))

    if best_epoch is not None:
        best_accuracy_val = accuracy_listVal[best_epoch]
    else:
        best_accuracy_val = 0
        best_epoch = 0
    model = model.to(device)
    model.train()
    accuracy = Accuracy(task="multiclass", num_classes=num_classes).to(device)

    for epoch in range(trained_epochs, num_epochs):
        running_loss = 0.0
        start_time = time.time()
        accuracy.reset() # ← manual reset

        # Probar a borrar para comprobar que no afecta
        # Validation Accuracte
        if dataloaderVal is not None:
            with torch.no_grad():
                for i, data in enumerate(dataloaderVal):
                    images, labels = data
                    images, labels = images.to(device), labels.to(device)
```

```

        outputs = model(images)
        preds = torch.argmax(outputs, dim=1)
        accuracy.update(preds, labels)
        epoch_accVal = accuracy.compute().item()
        accuracy.reset() # ← manual reset
    torch.cuda.empty_cache()
    torch.cuda.ipc_collect()
    gc.collect()
    # Train
    for i, data in enumerate(dataloader):
        images, labels = data
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    with torch.no_grad():
        preds = torch.argmax(outputs, dim=1)
        accuracy.update(preds, labels)
    torch.cuda.empty_cache()
    torch.cuda.ipc_collect()
    gc.collect()
    with torch.no_grad():
        epoch_acc = accuracy.compute().item()

epoch_loss = running_loss / len(dataloader)
losses[epoch] = epoch_loss
trained_time[epoch] = time.time() - start_time
accuracy_list[epoch] = epoch_acc
if dataloaderVal is not None:
    accuracy_listVal[epoch] = epoch_accVal
    accStr = f" Accuracy Val: {epoch_accVal:.4f} |"
else:
    accStr = ""

if ((torch.cuda.is_available() and ((torch.cuda.memory_allocated() / (1024**2)) > 10000)) or ((torch.cuda.memory_reserved() / (1024**2)) > 10000)):
    print("***** WARNING *****")
    print(f"CUDA memory allocated: {torch.cuda.memory_allocated() / 1024**2:.2f} MB")
    print(f"CUDA memory reserved: {torch.cuda.memory_reserved() / 1024**2:.2f} MB")

current_lr = optimizer.param_groups[0]['lr']
# END OF EPOCH
if scheduler is not None:
    scheduler.step()

# Save the best validation possible, no possibility of overfitting
if epoch_accVal > best_accuracy_val:

    best_accuracy_val = epoch_accVal
    best_epoch = epoch+1
    model = model.cpu()
    best_model_state_dict = copy.deepcopy(model.state_dict())
    model.to(device)
    best_model_values = [
        best_model_state_dict, # 0 model state
        copy.deepcopy(optimizer.state_dict()), # 1 optimizer
        copy.deepcopy(scheduler), # 2 scheduler
        None, # 3 epochs
        best_epoch, # 4 best_epoch
        None, # 5 losses
        None, # 6 model file name
        None, # 7 total trained time
        None, # 8 accuracy train list
        None # 9 accuracy val list
    ]
    print(f"\tBest version of model hit at epoch {epoch+1}, valAcc: {epoch_accVal}. parameters saved")
    torch.cuda.empty_cache()
    torch.cuda.ipc_collect()
    gc.collect()

```

```

if (((epoch+1) % ROUNDS_PER_SAVE_MODELS_CLASSIFIER == 0) or ((epoch+1) == num_epochs)):
    best_model_values[3] = epoch+1
    best_model_values[5] = losses
    best_model_values[6] = model_filename
    best_model_values[7] = trained_time
    best_model_values[8] = accuracy_list
    best_model_values[9] = accuracy_listVal
    save_classifier_model(*best_model_values)

    print(f"\tEpoch {epoch+1}/{num_epochs} Loss: {epoch_loss:.4f} | Accuracy train: {epoch_acc:.4f} | {accStr} LR actualizado:
    {current_lr:.8f} | Time: {trained_time[epoch]:.2f}s")
    print(f"\tModelo {model_filename} entrenado durante {num_epochs} épocas. Tiempo total: {sum(t for t in trained_time if t is not None):.2f}s")

```

12. Get best classifier and all classifiers accuracy

```

def get_bestModelClassifiers(models, trained_models_classifier, databases, dataloader_classifiersVal, dataloader_classifiersTrain,
filename):
    if CLASSIFIERS_ACCURACY:
        bestModel = {}
        if os.path.exists(filename+'.xlsx'):
            # Cargar DataFrame (multiíndice en columnas, primera columna como índice de modelos)
            df = pd.read_excel(filename+'.xlsx', header=[0, 1], index_col=0)
            # Eliminar columnas que se llamen 'Metric' si existen
            df = df.loc[:, df.columns.get_level_values(0) != 'Metric']
            # Ahora puedes acceder sin errores
            bestModel = {}
            for database in df.columns.levels[0]:
                # Asegúrate que la columna existe en ambos niveles
                if (database, 'val-1') in df.columns:
                    val1_column = df[(database, 'val-1')]
                    best_model_name = val1_column.idxmax()
                    best_model_value = val1_column.max()
                    bestModel[database] = {
                        'Model': best_model_name,
                        'val-1': best_model_value
                    }
            # Mostrar DataFrame cargado y mejor modelo
            print(df)
            print(bestModel)
            return bestModel

        # Calculate best model
        num = 1
        total = len(databases) * len(models)
        allmodelsAccuracy = {}
        allmodelsAccuracyPerClass = {}

        for database in databases:
            dataloaderVal = dataloader_classifiersVal[database]
            dataloaderTrain = dataloader_classifiersTrain[database]

            bestModel[database] = None
            allmodelsAccuracy[database] = {}
            allmodelsAccuracyPerClass[database] = {}
            bestModelAcc = 0
            print(f"***** {database} *****")

            for modelName in models.keys():
                print(f"{num}/{total} - ModelName: {modelName} - {database}")
                allmodelsAccuracy[database][modelName] = {}
                allmodelsAccuracyPerClass[database][modelName] = {}

            for name in trained_models_classifier[database]:
                name = os.path.basename(name)
                if modelName in name:
                    path = os.path.join(MODELS_CLASSIFIERS, name)
                    if os.path.isfile(path):
                        # Cargar modelo
                        checkpoint = torch.load(path)
                        model, lr = copy.deepcopy(models[modelName])

```

```

best_epoch = checkpoint['best_epoch']
model.load_state_dict(checkpoint['model_state_dict'])
model.to(device)
model.eval()

### Por Clase
# Inicializar contadores
correct_per_class = defaultdict(int)
total_per_class = defaultdict(int)

total_correct = 0
total_samples = 0

with torch.no_grad():
    for images, labels in dataloaderVal:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        preds = torch.argmax(outputs, dim=1)

        for label, pred in zip(labels, preds):
            label_id = label.item()
            total_per_class[label_id] += 1
            total_samples += 1
            if pred.item() == label_id:
                correct_per_class[label_id] += 1
                total_correct += 1

# Calcular precisión por clase
accuracy_per_class = {}
for class_id in total_per_class:
    accuracy = correct_per_class[class_id] / total_per_class[class_id]
    accuracy_per_class[class_id] = accuracy

# Mostrar precisión por clase
for class_id in sorted(accuracy_per_class.keys()):
    # print(f"Clase {class_id}: {accuracy_per_class[class_id]*100:.2f}% acierto")

# Mostrar precisión total
accuracy_total = total_correct / total_samples
# print(f"\nPrecisión total del modelo: {accuracy_total*100:.2f}%")
###

# Inicializar métricas Top-1 y Top-5
accuracy_top1 = Accuracy(task="multiclass", num_classes=10, top_k=1).to(device)
accuracy_top5 = Accuracy(task="multiclass", num_classes=10, top_k=5).to(device)

## Validation Accuracy
with torch.no_grad():
    for images, labels in dataloaderVal:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        # Top-1
        preds_top1 = torch.argmax(outputs, dim=1)
        accuracy_top1.update(preds_top1, labels)
        # Top-5
        accuracy_top5.update(outputs, labels) # outputs sin argmax

# Resultados finales
final_accuracyVal_top1 = accuracy_top1.compute().cpu().item()
final_accuracyVal_top5 = accuracy_top5.compute().cpu().item()
# Reset métricas antes del train
accuracy_top1.reset()
accuracy_top5.reset()

## Train Accuracy
with torch.no_grad():
    for images, labels in dataloaderTrain:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        preds_top1 = torch.argmax(outputs, dim=1)
        accuracy_top1.update(preds_top1, labels)
        accuracy_top5.update(outputs, labels)

```

```

final_accuracyTrain_top1 = accuracy_top1.compute().cpu().item()
final_accuracyTrain_top5 = accuracy_top5.compute().cpu().item()

print(f"\tAccuracy %:"
      f"\n\t - Validation: Top-1: {final_accuracyVal_top1:.4f} | Top-5: {final_accuracyVal_top5:.4f}"
      f"\n\t - Train:   Top-1: {final_accuracyTrain_top1:.4f} | Top-5: {final_accuracyTrain_top5:.4f}")

allmodelsAccuracy[database][modelName] = [final_accuracyTrain_top1, final_accuracyVal_top1,
final_accuracyTrain_top5, final_accuracyVal_top5, best_epoch]
allmodelsAccuracyPerClass[database][modelName] = accuracy_per_class

if final_accuracyVal_top1 > bestModelAcc:
    bestModelAcc = accuracy_total
    bestModel[database] = [model.cpu(), name, bestModelAcc, accuracy_per_class]
del model, lr
num += 1
print()

print("Showing accuracy per class all models")
print(allmodelsAccuracyPerClass)

for database in databases:
    print(f"Best Model {database}: ", bestModel[database][1], bestModel[database][2], bestModel[database][3])
    plot_accuracy_per_class_grouped(allmodelsAccuracyPerClass, database, filename+fPerClass_{database}.png')

dict_to_dataframe(allmodelsAccuracy, ['train-1', 'val-1', 'train-5', 'val-5', 'epoch'], filename+'.xlsx')
print('Saved metrics')

return bestModel

```

13. Get accuracy of classifiers using DCGAN images

IMAGES_PER_CLASS = 395 # Images in dataloader of real set

```

class ImagenesGeneradasDataset(Dataset):
    def __init__(self, imgs, labels, transform):
        self.preloaded_data = []

        # Transform interno: Resize a 224x224 y conversión a tensor
        self.transform = transform

    for img, label in zip(imgs, labels):
        # img es tensor en este punto, conviértelo primero a PIL Image
        img_pil = to_pil_image(img)
        img_resized = self.transform(img_pil)
        self.preloaded_data.append((img_resized, label))

    def __len__(self):
        return len(self.preloaded_data)

    def __getitem__(self, idx):
        return self.preloaded_data[idx]

def getBestDCGANGeneratorsDCGAN(trained_models, classifierModel, trainedClassifiersFileName, filename):
    accuracy_fake_images = {
        # Key: database
        # value: dict{
        #   Key: image_Size
        #   value: dict{
        #     key: n (name of the classifier)
        #     value: [
        #       (modelG, lrD=0.01, lrG=0.02, % top 1 - right classified images, % top 5)
        #     ]
        #   }
        # }
        # }
    }

    # Initialization for no error in future functions
    bestModelForEachClass = {
        IMAGENETTE: {x:(-1, "") for x in classifierImagenette_inv},
        IMAGEWOOF: {x:(-1, "") for x in classifierImagewoof_inv}
    }

```

```

}
if CLASSIFIERS_ACCURACY and TRAIN_DCGAN:

    filenameImagenette = f'{os.path.join(PLOTS, filename)}_{IMAGENETTE}.xlsx'
    filenameImagewoof = f'{os.path.join(PLOTS, filename)}_{IMAGEWOOF}.xlsx'

    if os.path.exists(filenameImagenette) and os.path.exists(filenameImagewoof):
        print('Accuracy already calculated in', filename)
        return None

    total = 0
    for database in DATABASES:
        for img_size in IMG_SIZES:
            total += len(trained_models[(img_size, database)])
    print("Total Models: ", total)

    modelCount = 0
    for database in DATABASES:
        path = os.path.join(MODELS_CLASSIFIERS, trainedClassifiersFileName[database])
        print(path)
        if os.path.isfile(path):
            # Cargar modelo
            checkpoint = torch.load(path)
            classifierModel = copy.deepcopy(classifierModel)
            classifierModel.load_state_dict(checkpoint['model_state_dict'])
            classifierModel.to(device)
            classifierModel.eval()
        else:
            print("no se encontró el clasificador")

    print(f"DATABASE: {database}")
    if database not in accuracy_fake_images: # Iniciar el dict
        accuracy_fake_images[database] = {}
    for img_size in IMG_SIZES:
        print(f"IMAGE SIZE: {img_size}")
        if img_size not in accuracy_fake_images[database]:
            accuracy_fake_images[database][img_size] = {}
        for num, model in enumerate(trained_models[(img_size, database)].keys(), 1):
            # Modelo referencia
            modelCount += 1
            print(f"Model {num} - {modelCount}/{total}")
            for pair_model in trained_models[(img_size, database)][model]: # Modelo para cada clase
                modelD_filename, modelG_filename, datasetTitle = pair_model
                if os.path.isfile(modelD_filename) and os.path.isfile(modelG_filename):
                    patron = r"^(.)*(?=_Model)"
                    coincidencia = re.search(patron, modelG_filename)

                    if coincidencia:
                        modelName = os.path.basename(coincidencia.group(1))
                    else:
                        print("La cadena no coincide con el patrón.")
                        continue

                pattern = r"nz=(\d+)_lrD=(\d+)_lrG=(\d+)_imgSize=(\d+)_uD=(\d+)_uG=(\d+)_modData=(\d+)_ModelG-([a-z0-9]+)"
                match = re.match(pattern, os.path.basename(modelG_filename))

                if match:
                    nz = int(match.group(1))
                    lrD = float(match.group(2))
                    lrG = float(match.group(3))
                    imgSize = int(match.group(4))
                    uD = int(match.group(5))
                    uG = int(match.group(6))
                    modData = int(match.group(7))
                    n = match.group(8)
                else:
                    print("La cadena no coincide con el patrón.")
                    continue

                if imgSize != img_size:
                    print("ALGO RARO PASA")
                    continue

                if n not in accuracy_fake_images[database][img_size]:
                    accuracy_fake_images[database][img_size][n] = []

```



```

# Cargar modelo
netG = copy.deepcopy(generators_dict[imgSize][nz])
checkpoint = torch.load(modelG_filename)
netG.load_state_dict(checkpoint['model_state_dict'])

# Generar imagenes
netG = netG.to(device)
fakeImages = netG(noise_for_accuracy[nz]).detach().cpu()
fakeImages = (fakeImages + 1) / 2

if database == IMAGENETTE:
    label = id_to_index_imagenette[n]
elif database == IMAGEWOOF:
    label = id_to_index_imagewoof[n]
else:
    print("ALGO RARO PASA")
    continue
print(f"\t\t\tClase {label}")

labels = torch.full((IMAGES_PER_CLASS,), label, dtype=torch.long, device=device)

# Crear Dataset de cada clase para el modelo con estos parametros
dataset = ImagenesGeneradasDataset(fakeImages, labels, TRANSFORM_VAL)

# Crear Dataloader
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE_CLASSIFIER, shuffle=True)

# Ver el % exito de clasificación del mejor modelo
# Inicializar métricas Top-1 y Top-5
accuracy_top1 = Accuracy(task="multiclass", num_classes=10, top_k=1).to(device)
accuracy_top5 = Accuracy(task="multiclass", num_classes=10, top_k=5).to(device)

## Validation Accuracy
with torch.no_grad():
    for images, labels in dataloader:
        images, labels = images.to(device), labels.to(device)
        outputs = classifierModel(images)

        # Top-1
        preds_top1 = torch.argmax(outputs, dim=1)
        accuracy_top1.update(preds_top1, labels)

        # Top-5
        accuracy_top5.update(outputs, labels) # outputs sin argmax

# Resultados finales
final_accuracyVal_top1 = accuracy_top1.compute().item()
final_accuracyVal_top5 = accuracy_top5.compute().item()

modelG_filename = f"lrD={lrD}, lrG={lrG}, modData={modData}"

if bestModelForEachClass[database][n][0] < final_accuracyVal_top1:
    #print(f"Updated accuracy for DB:{database} - class:{n}, from {bestModelForEachClass[database][n][0]} to {final_accuracyVal_top1}")
    bestModelForEachClass[database][n] = (float(f"{final_accuracyVal_top1:.4f}"), modelG_filename)

print(f"\t\t\tAccuracy % - {classifierFromNameInv[database][n]} ({database}):"
      f"\n\t\t\t\t\tValidation: Top-1: {final_accuracyVal_top1:.4f} | Top-5: {final_accuracyVal_top5:.4f}")

accuracy_fake_images[database][img_size][n].append((modelG_filename, final_accuracyVal_top1,
final_accuracyVal_top5))
del accuracy_top1, accuracy_top5
print("Best Models: ")
print(bestModelForEachClass)

print("\nAll models")
print(accuracy_fake_images)

```

```

def create_dataframe(database_name, class_dict_inv, filename):
    data = {}
    def highlight_max(s):
        is_max = s == s.max()
        return ['text-decoration: underline;' if v else '' for v in is_max]

    for img_size, labels in accuracy_fake_images[database_name].items():
        for label, records in labels.items():
            class_name = class_dict_inv[label]
            for modelG_filename, acc_top1, _ in records:
                if modelG_filename not in data:
                    data[modelG_filename] = {}
                data[modelG_filename][class_name] = f'{acc_top1:.2f}'

    df = pd.DataFrame.from_dict(data, orient='index').reset_index().rename(columns={'index': 'Model_Config'})
    df_styled = df.style.apply(highlight_max, subset=pd.IndexSlice[:, df.columns[1:]])
    df_styled.to_excel(filename, index=False, engine='openpyxl')
    return df

# Crear DataFrames separados
create_dataframe(IMAGENETTE, classifierImagenette_inv, filenameImagenette)
create_dataframe(IMAGEWOOF, classifierImagewoof_inv, filenameImagewoof)

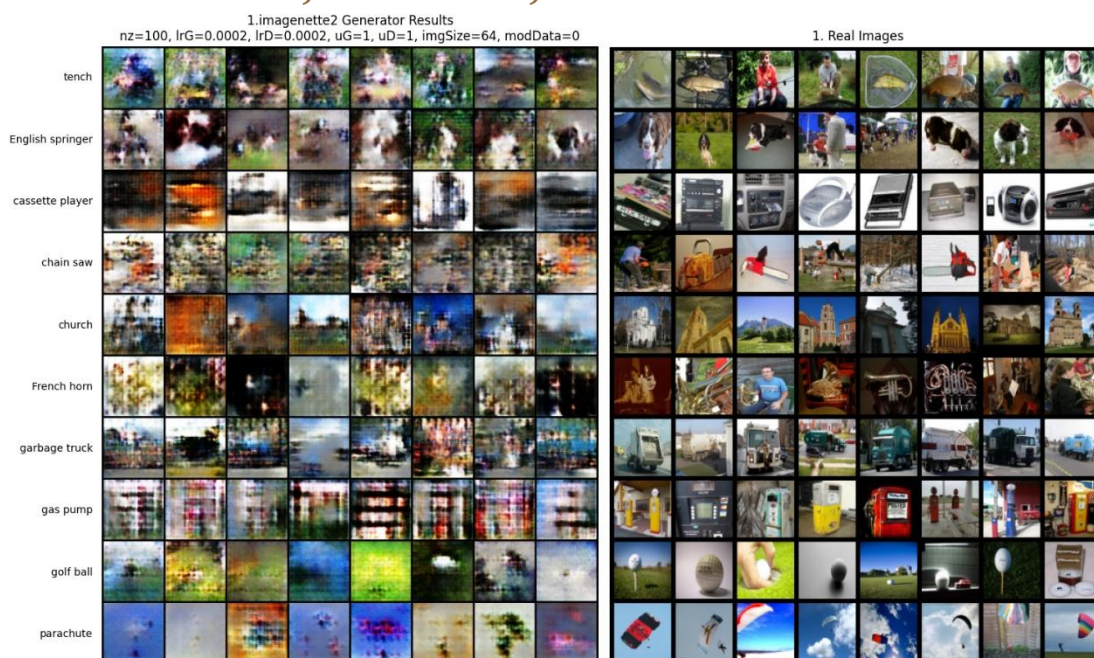
return bestModelForEachClass

```

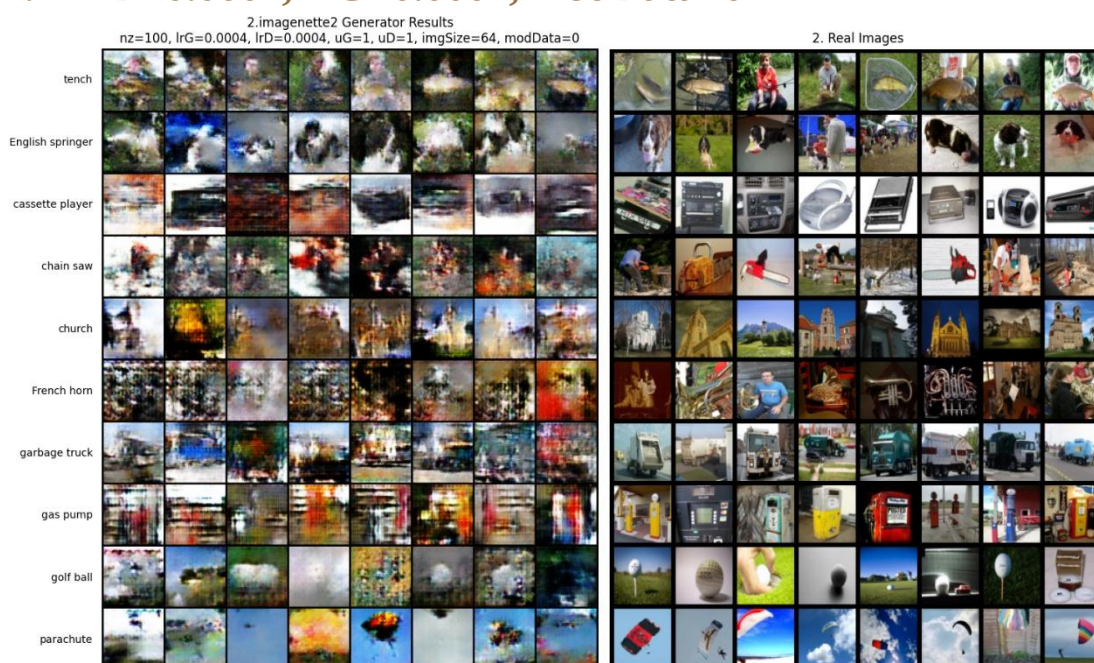
DCGAN Results

We will visualize here the final results of the image creations of the 6 model configurations:

1. lrD=0.0002, lrG=0.0002, modData=0



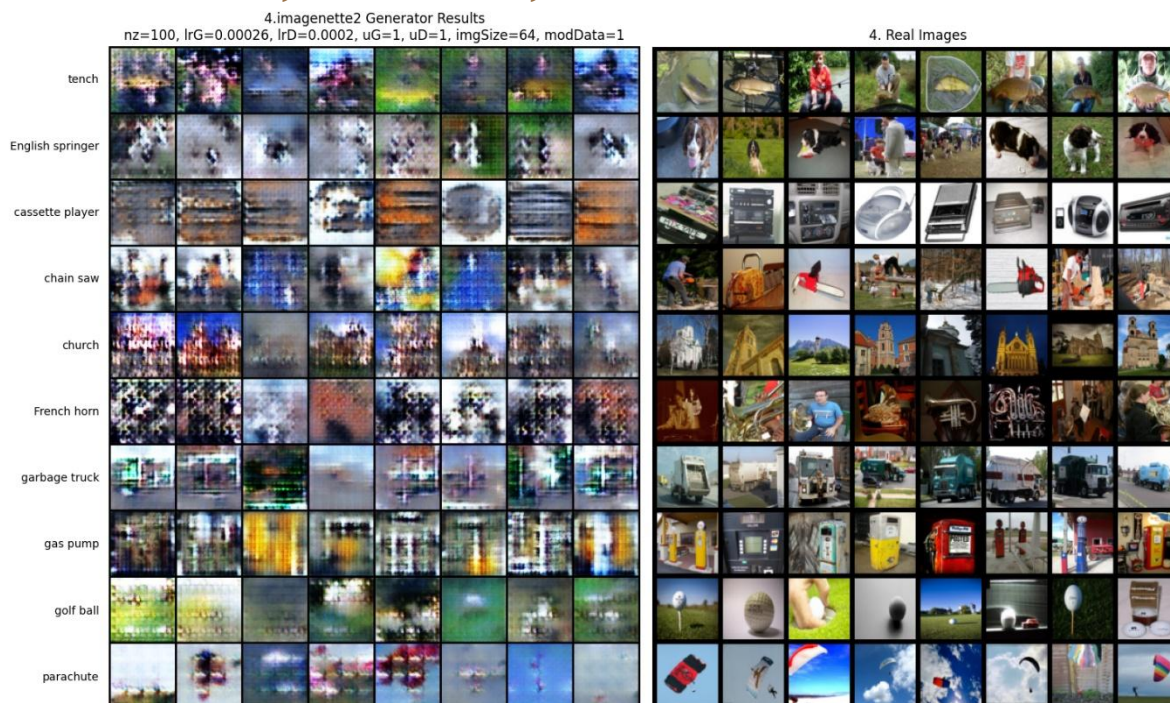
2. lrD=0.0004, lrG=0.0004, modData=0



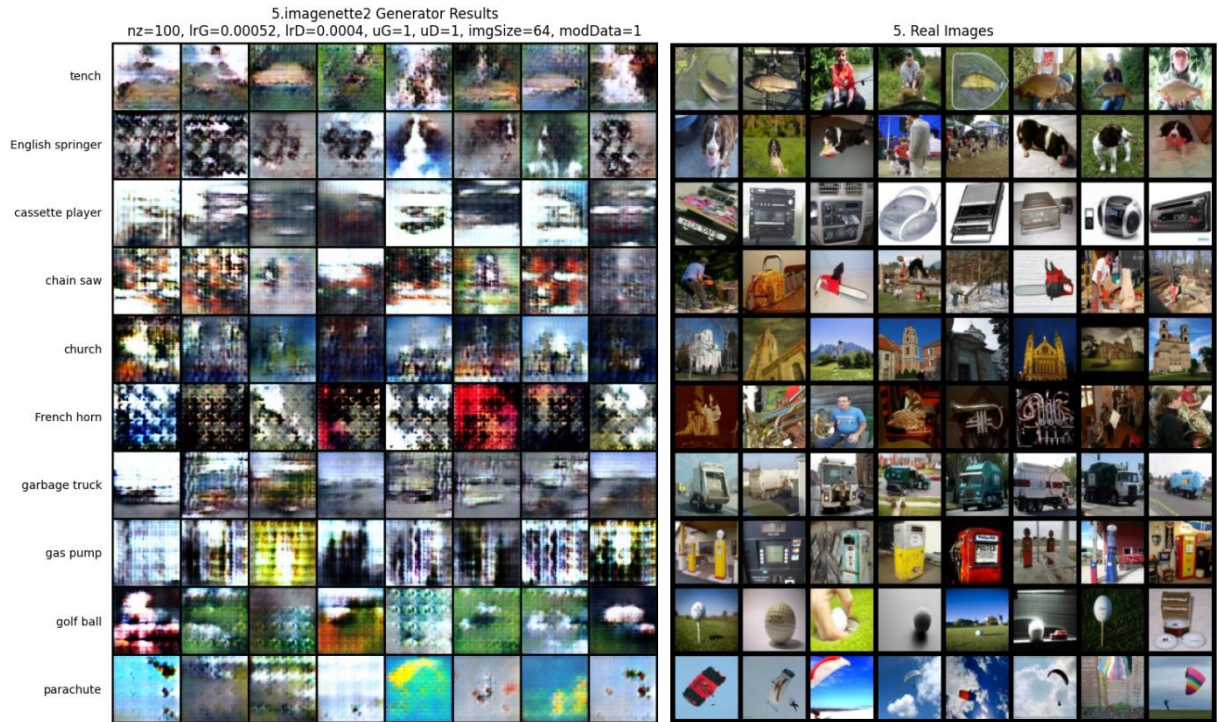
3. $lrD=0.0006$, $lrG=0.0006$, $modData=0$



4. $lrD=0.0002$, $lrG=0.00026$, $modData=1$



5. $\text{lrD}=0.0004$, $\text{lrG}=0.00052$, $\text{modData}=1$



6. $\text{lrD}=0.0006$, $\text{lrG}=0.00078$, $\text{modData}=1$

